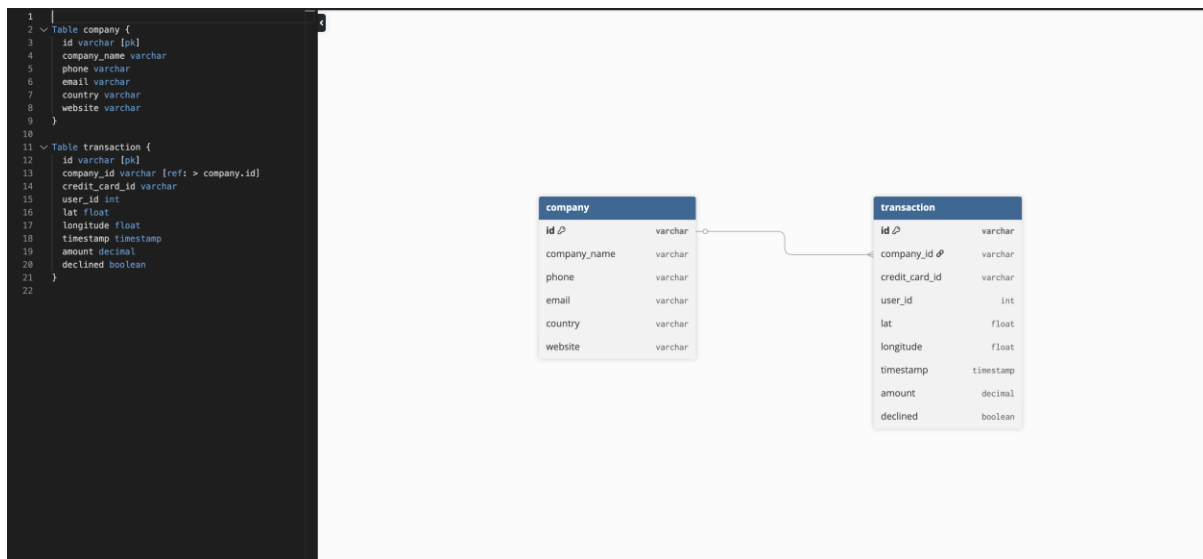


SPRINT 2 – DATA ANALYTICS 2026

LEONARDO ANTONIO RUGGIERO

Ejercicio 1

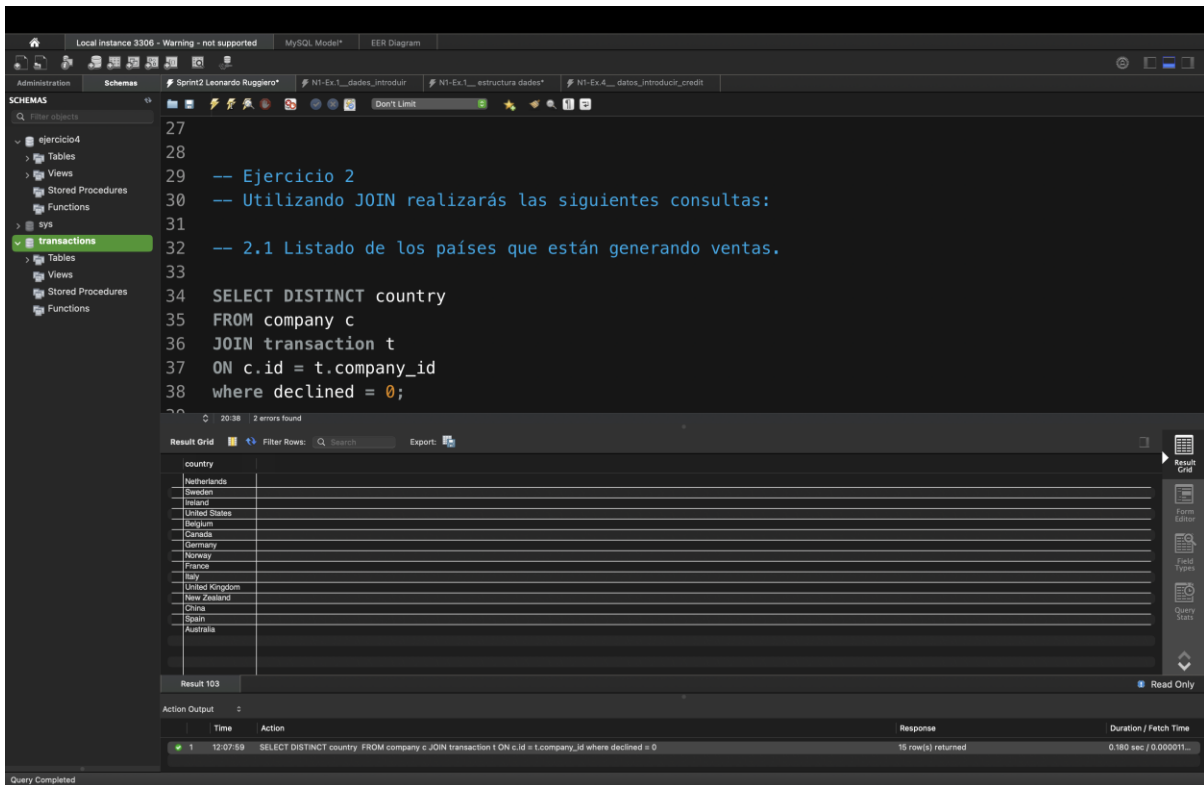
-1.1: A partir de los documentos adjuntos (estructura_datos y datos_introducir), importa las dos tablas. Muestra las principales características del esquema creado y explica las diferentes tablas y variables que existen. Asegúrate de incluir un diagrama que ilustre la relación entre las distintas tablas y variables.



Respuesta:

Para este ejercicio, empecé preparando el entorno en **MySQL Workbench**. Lo primero que hice fue cargar las dos bases de datos importando sus respectivos **scripts SQL** y ejecutándolos directamente en mi editor. Este paso fue fundamental para tener todas las tablas y registros listos antes de empezar con las modificaciones. En mi estructura, tengo una tabla central de hechos llamada **transaction** y una tabla de dimensiones llamada **company**. En la tabla de hechos, manejo información clave como el ID de cada venta, si fue denegada o no, los datos de las tarjetas, el usuario que realizó la compra, el importe y la ubicación geográfica mediante latitud y longitud. La relación entre ambas tablas se establece mediante la **Foreign Key business_id** (en la tabla transaction), que se conecta con la **Primary Key company_id** de la tabla company. Esta relación es de **1:N** (uno a muchos), lo que significa que mientras cada empresa aparece una sola vez en su tabla de dimensiones, puede aparecer repetidas veces en la tabla de hechos porque una misma empresa puede generar múltiples transacciones. Para visualizar esta arquitectura, utilicé la herramienta **dbdiagram.io**. Inserté el código de mis tablas en esta plataforma para generar el diagrama relacional de forma limpia y profesional, lo que me permitió confirmar que las conexiones entre hechos y dimensiones fueran correctas.

Ejercicio 2.1



The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'ejercicio4' selected, containing 'Tables', 'Views', 'Stored Procedures', and 'Functions'. The 'transactions' table is highlighted. The main editor shows a SQL query:

```
27
28
29 -- Ejercicio 2
30 -- Utilizando JOIN realizarás las siguientes consultas:
31
32 -- 2.1 Listado de los países que están generando ventas.
33
34 SELECT DISTINCT country
35 FROM company c
36 JOIN transaction t
37 ON c.id = t.company_id
38 where declined = 0;
```

The 'Result Grid' at the bottom shows the query results for 'Result 103'. The results are as follows:

country
Netherlands
Sweden
Ireland
United States
Belgium
Colombia
Germany
Norway
France
Italy
United Kingdom
New Zealand
China
Spain
Australia

The 'Action Output' pane at the bottom shows the query execution details:

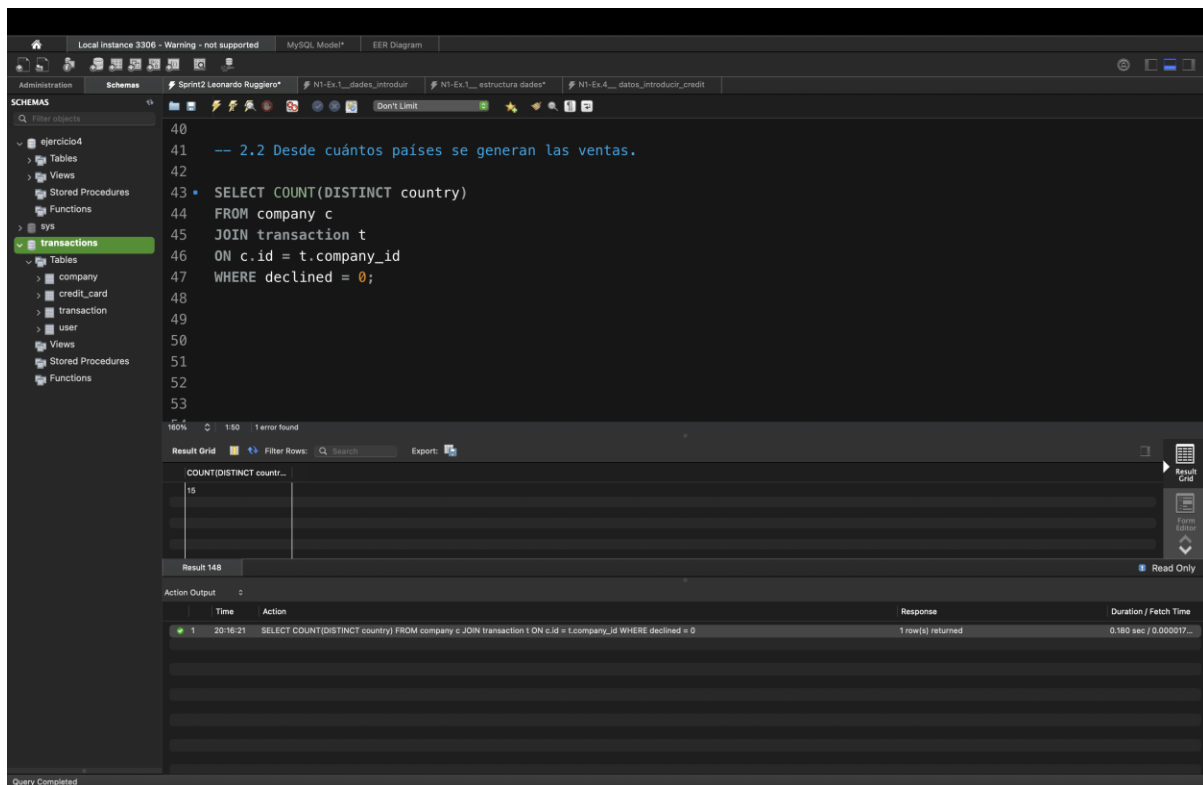
Time	Action	Response	Duration / Fetch Time
12:07:59	SELECT DISTINCT country FROM company c JOIN transaction t ON c.id = t.company_id where declined = 0	15 row(s) returned	0.180 sec / 0.000011...

Query Completed

Respuesta:

Las relaciones del “JOIN” siempre serán entre el campo “ID” de la tabla “company” y el campo “Company_ID” de la tabla “transaction”. Una vez que hemos creado esa relación entre las dos tablas, en este caso lo que hemos hecho es seleccionar el campo de country de la tabla “company” con un “distinct”, porque no queremos que ninguno de los valores que tenemos nos salgan repetidos. Por fin he añadido “WHERE declined = 0” para que en la salida solo aparezcan transacciones aprobadas.

Ejercicio 2.2



The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'ejercicio4' selected, and 'transactions' expanded under 'Tables'. The main editor window contains the following SQL query:

```
40
41 -- 2.2 Desde cuántos países se generan las ventas.
42
43 • SELECT COUNT(DISTINCT country)
44   FROM company c
45  JOIN transaction t
46   ON c.id = t.company_id
47   WHERE declined = 0;
48
49
50
51
52
53
```

Below the query editor, the 'Result Grid' shows the query results:

COUNT(DISTINCT countr...
15

The 'Action Output' pane at the bottom shows the execution details:

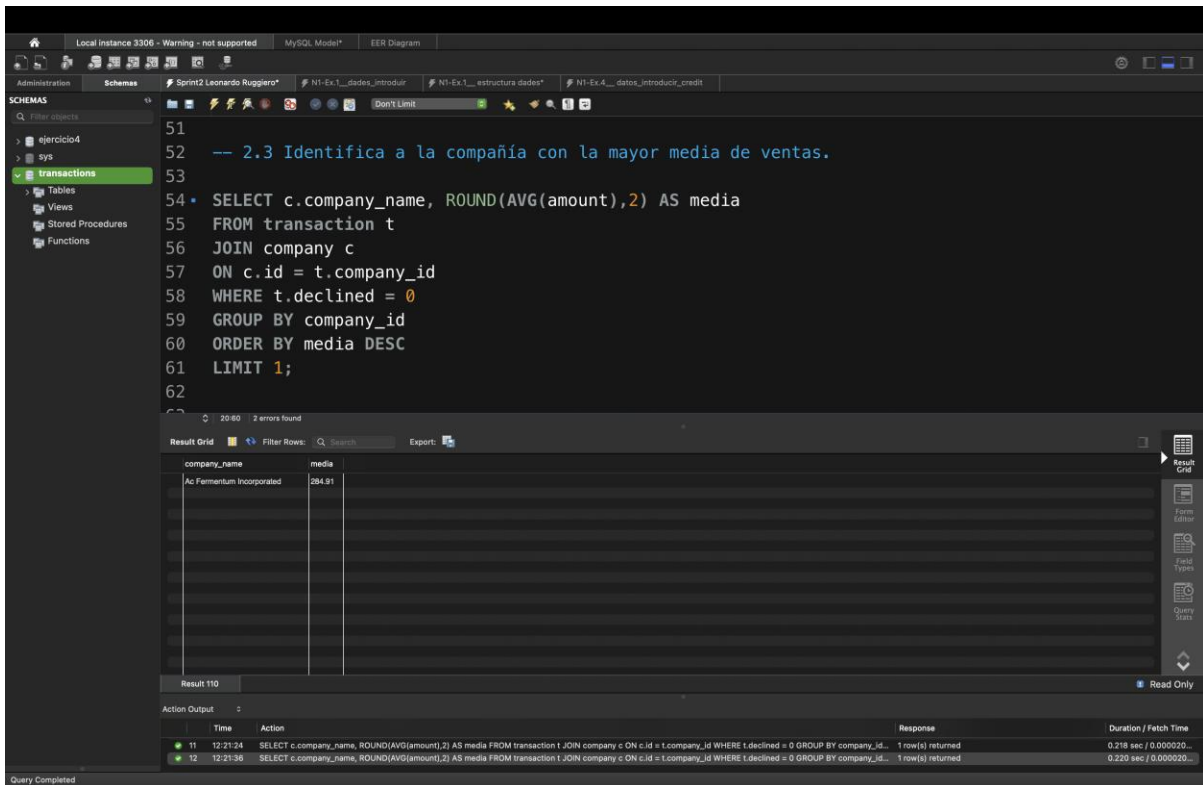
Time	Action	Response	Duration / Fetch Time
20:16:21	SELECT COUNT(DISTINCT country) FROM company c JOIN transaction t ON c.id = t.company_id WHERE declined = 0	1 row(s) returned	0.180 sec / 0.000017...

The status bar at the bottom indicates 'Query Completed'.

Respuesta:

Utilizo la función de agregación “**COUNT(DISTINCT country)**” para obtener el número total de países únicos que han registrado ventas. El uso de **DISTINCT** dentro del conteo es indispensable para asegurar que cada país se contabilice una sola vez, independientemente del volumen de transacciones que tenga asociado. Por fin he añadido “**WHERE declined = 0**” para que en la salida solo aparezcan transacciones aprobadas.

Ejercicio 2.3



The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
51
52 -- 2.3 Identifica a la compañía con la mayor media de ventas.
53
54 * SELECT c.company_name, ROUND(AVG(amount),2) AS media
55 FROM transaction t
56 JOIN company c
57 ON c.id = t.company_id
58 WHERE t.declined = 0
59 GROUP BY company_id
60 ORDER BY media DESC
61 LIMIT 1;
62
```

The Results tab shows the following data:

company_name	media
Ac Fermentum Incorporated	284.91

The Action Output tab shows the execution details:

Time	Action	Response	Duration / Fetch Time
11 12:21:24	SELECT c.company_name, ROUND(AVG(amount),2) AS media FROM transaction t JOIN company c ON c.id = t.company_id WHERE t.declined = 0 GROUP BY company_id...	1 row(s) returned	0.218 sec / 0.000020...
12 12:21:38	SELECT c.company_name, ROUND(AVG(amount),2) AS media FROM transaction t JOIN company c ON c.id = t.company_id WHERE t.declined = 0 GROUP BY company_id...	1 row(s) returned	0.220 sec / 0.000020...

Respuesta:

Utilizo la función “**AVG**” junto con “**GROUP BY**” para calcular el promedio de ventas por empresa. Aplico “**ORDER BY DESC**” y “**LIMIT 1**” para aislar únicamente el registro con el valor más alto, cumpliendo con el objetivo de identificar la compañía con la mayor media. Además, empleo **ROUND(..., 2)** para cumplir con la normativa de formato para valores monetarios. Por fin he añadido “**WHERE declined = 0**” para que en la salida solo aparezcan transacciones aprobadas.

Ejercicio 3.1

The screenshot shows a MySQL IDE interface with a SQL query editor and a results grid. The query is as follows:

```
32
33 -- Utilizando sólo subconsultas (sin utilizar JOIN):
34
35 -- 3.1 Muestra todas las transacciones realizadas por empresas de Alemania.
36
37 * SELECT *
38 FROM transaction t
39 WHERE EXISTS (SELECT company_id FROM company c WHERE c.id = t.company_id AND c.country = 'Germany' AND t.declined = 0);
40
```

The results grid displays the following columns: id, credit_card..., company_id, user_id, lat, longitude, timestamp, amount, and declined. The results show a list of transactions for companies in Germany, with columns truncated for brevity.

id	credit_card...	company_id	user_id	lat	longitude	timestamp	amount	declined
00138038-20D-4C03-9487-63A267E9B964	C6S-4889	b-2222	318	41.3781	12.447	2020-03-25 10:43:43	426.36	0
0018C186-38A-4D0C-8154-E2B8F80CA8B9	C6S-5010	b-2222	480	41.3814	12.1876	2020-12-17 18:12:37	218.90	0
00201A11-2E62-44C4-941D-196F-C8D877F9	C6U-3512	b-2222	185	55.5704	3.68129	2021-01-22 23:44:27	433.04	0
00235818-0A3C-4D49-90C8-63A842D0B923	C6S-8137	b-2222	3058	58.8421	18.729	2020-09-09 15:43:19	263.14	0
00245419-31A-4852-9815-162476D0C28	C6S-8988	b-2222	4417	51.1581	8.68905	2024-02-15 09:10:11	142.0	0
00887139-4B82-4FFA-8E73-820378F04A84	C6S-4870	b-2222	289	51.1866	10.4869	2019-03-09 19:37:49	524.84	0
0074F4D0-3D71-4827-8758-0498314623A	C6S-4061	b-2222	3500	56.7016	4.56325	2016-12-26 23:06:57	491.90	0
00A8B0C0-36C8-4D2B-8A10-138E72D0C9AB	C6S-4787	b-2222	2218	55.7852	3.78245	2021-04-29 03:56:59	167.15	0
00E0804-8920-4708-ABE5-32E2268626D	C6S-4883	b-2222	402	58.708	8.12993	2019-02-27 15:25:18	141.66	0
00DA0383-E548-4377-8ED1-3C5C256FF2F	C6S-9223	b-2222	4642	51.1742	10.2627	2019-03-21 11:47:34	325.62	0
00D511D5-E0D1-48AD-33A3-17AD-1E8A50F	C6S-7661	b-2222	3109	45.7649	4.83109	2024-07-28 18:30:49	242.53	0
01449C6D-98E9-4DE3-9810-798C8A6056F	C6S-5424	b-2222	643	47.0163	2.26054	2024-08-17 19:37:14	451.71	0
0176E8C7-241E-42DA-A889-9F240DF4D26	C6S-7510	b-2222	2629	52.0019	4.29484	2021-08-28 16:20:38	9.46	0
01A8DA0B-06E2-42A0-A131-AEE6FF11B789	C6S-5583	b-2222	416	51.7438	5.17479	2020-01-28 01:15:07	368.43	0
01F1C7ED-0823-442D-AE5E-3134D5D04866	C6S-6776	b-2222	2195	58.6697	18.6697	2022-12-17 09:40:14	168.79	0
01FAB051-1B08-441B-98FC-428A3F06097	C6S-5831	b-2222	965	55.3405	-3.3863	2020-04-30 08:57:27	333.48	0
0203714C-5D01-4502-A9E6-2865874F61B	C6S-5103	b-2222	292	45.5382	2.99198	2019-04-14 15:52:13	155.59	0
0248B812-89F5-4B21-B8E9-E0D51800A579	C6S-9328	b-2222	4747	42.0816	12.8338	2016-07-01 01:33:41	268.23	0
02DF683C-E822-42D3-B78E-15A0A3A27694	C6S-4889	b-2222	276	60.8186	18.5278	2021-02-03 14:12:33	127.23	0
0310290E-F9A2-4448-95EE-87000086534D	C6U-5760	b-2222	189	55.355	3.44611	2020-04-05 19:32:29	68.38	0

The bottom of the screenshot shows the "Action Output" section with the following entry:

Time	Action	Response	Duration / Fetch Time
12:35:00	SELECT * FROM transaction t WHERE EXISTS (SELECT company_id FROM company c WHERE c.id = t.company_id AND c.country = 'Germany' AND t.declined = 0)	13289 rows returned	0.0017 sec / 0.066 sec

Respuesta:

Siguiendo el requerimiento del enunciado de no utilizar JOIN, empleo una **subconsulta** sobre la tabla company para recuperar los identificadores (id) asociados a Alemania. Utilizo este conjunto de datos como filtro en la cláusula **“WHERE EXISTS”** de la consulta principal sobre transaction. Adicionalmente, he incluido la condición “declined = 0” para asegurar que el listado solo muestre transacciones efectivas, excluyendo aquellas que fueron declinadas, optimizando así la relevancia de los datos obtenidos.

Ejercicio 3.2

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'ejercicio4' selected, and the 'transactions' table highlighted under the 'company' schema. The main editor window contains the following SQL query:

```
74
75 -- 3.2 Lista las empresas que han realizado transacciones por un amount superior a la media de todas las transacciones
76
77 SELECT c.company_name
78 FROM company as c
79 WHERE EXISTS (
80     SELECT id
81     FROM transaction as t
82     WHERE t.company_id = c.id AND t.amount > (SELECT AVG(amount) FROM transaction) AND t.declined = 0);
83
84
85
86
87
```

Below the query editor, the 'Result Grid' shows the results of the query. The first column is 'company_name' and the second column is 'company_id'. The results are as follows:

company_name	company_id
Ac Fomentum Incorporated	
Magna A Neque Industries	
Fusce Corp	
Consectetur Inc Incorporated	
Arto laculis Nec Foundation	
Donec Ltd	
Ipsum Nunc Ltd	

The 'Action Output' pane at the bottom shows the execution details of the query:

Time	Action	Response	Duration / Fetch Time
1	20:19:21 SELECT c.company_name FROM company as c WHERE EXISTS (SELECT id FROM transaction as t WHERE t.company_id = c.id AND t.amount > (SELECT AVG(amo...	100 row(s) returned	0.077 sec / 0.000022...

Respuesta:

Para resolver este ejercicio sin utilizar “JOIN”, empleo la cláusula “**WHERE EXISTS**” con una **subconsulta correlacionada**. Esta estructura permite filtrar las empresas (company) basándose en la existencia de al menos una transacción en la tabla transaction que cumpla con los criterios especificados. Dentro de la subconsulta, comparo el importe de cada transacción (t.amount) contra una **subconsulta escalar** que calcula el promedio global (AVG(amount)) de toda la tabla. He decidido mantener la condición t.declined = 0 para asegurar que el cálculo y el filtrado se realicen exclusivamente sobre transacciones exitosas, garantizando la integridad de la lógica de negocio solicitada.

Ejercicio 3.3

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'ejercicio4' selected, containing tables like 'company', 'credit_card', 'transaction', and 'user'. The main editor shows a SQL query:

```
86
87
88
89 -- 3.3 Eliminarán del sistema las empresas que carecen de transacciones registradas, entrega el listado de estas
90 -- empresas.
91
92 * SELECT *
93 FROM company c
94 WHERE NOT EXISTS ( SELECT id FROM transaction t WHERE t.company_id = c.id AND t.declined = 1);
95
96
97
98
99
```

Below the query, the 'Result Grid' shows 156 rows of data with columns: id, company_name, phone, email, country, and website. The first few rows are:

id	company_name	phone	email	country	website
1	Mus Aenean Eget Foundation	06 25 15 52 43	mi.duis@hotmail.net	Sweden	https://instagram.com/group9
2	Dis Parturient Institute	05 36 29 19 74	janice@protonmail.org	Ireland	https://google.com/one
3	Eit Exam Limited Associates	07 69 75 17 45	dfrose@google.co.uk	Canada	https://yahoo.com/
4	Nunc Intendum Incorporated	05 18 15 48 13	honi@outlook.com	Germany	https://wikipedia.org/en-us
5	A Institute	03 34 91 68 65	melus.alquari@google.edu	Belgium	https://reddit.com/
6	Integer Molis Corp.	03 12 50 45 54	ita.eros@protonmail.ca	Italy	https://mellit.com/group9
7	Amet Institute	06 33 40 21 33	hulam.loboris.quam@outlook.net	Australia	https://nytimes.com/one

The 'Action Output' pane at the bottom shows two successful queries:

	Time	Action	Response	Duration / Fetch Time
1	20:24:34	SELECT * FROM company c WHERE NOT EXISTS (SELECT id FROM transaction t WHERE t.company_id = c.id AND t.declined = 1)	14 row(s) returned	0.034 sec / 0.000014...
2	20:26:27	SELECT * FROM company c WHERE NOT EXISTS (SELECT id FROM transaction t WHERE t.company_id = c.id AND t.declined = 1)	14 row(s) returned	0.034 sec / 0.000017...

Respuesta:

Utilize la cláusula **EXISTS** con una **subconsulta correlacionada** para filtrar la tabla company. He vinculado ambas tablas mediante la condición `t.company_id = c.id` dentro de la subconsulta para verificar qué empresas tienen registros asociados en la tabla de transacciones. He aplicado el filtro `declined = 1` para limitar el resultado únicamente a aquellas empresas que presentan transacciones declinadas.

Ejercicio 4

Local instance 3306 - Warning - not supported MySQL Mode*

Administration Schemas Sprint2 Leonardo Ruggiero*

SCHEMAS

Filter objects

sys

transactions

Tables

company

transaction

user

Views

Stored Procedures

Functions

55 -- Ejercicio 4

56

57 -- Tu tarea es diseñar y crear una tabla llamada "credit_card" que almacene detalles cruciales sobre las tarjetas de crédito.

58 -- La nueva tabla debe ser capaz de identificar de forma única cada tarjeta y establecer una relación adecuada con las otras dos tablas

59 -- ("transaction" y "company"). Después de crear la tabla será necesario que ingreses la información del documento

60 -- denominado "datos_introducir_credit". Recuerda mostrar el diagrama y realizar una breve descripción del mismo.

61

62

63 CREATE TABLE credit_card (

64 id VARCHAR(15) PRIMARY KEY,

65 iban VARCHAR(50),

66 pan VARCHAR(20),

67 pin VARCHAR(4),

68 cvv VARCHAR(3),

69 expiring_date VARCHAR(10)

70);

71

72

73

74

75

76

77

78

79

80

0.1

1:81 2 errors found

Result Grid

Filter Rows: Search Edit Export/Import

id	credLCard...	company_id	user_id	lat	longitude	timestamp	amount	declined
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

transaction 143

Action Output

Time	Action	Response	Duration / Fetch Time
52... 13:12:05	CREATE TABLE credit_card (id VARCHAR(15) PRIMARY KEY, iban VARCHAR(50), pan VARCHAR(20), pin VARCHAR(4), cvv VARCHAR(3), expiring_date VARCHAR(10)	0 row(s) affected	0.0079 sec

Query Completed

Local instance 3306 - Warning - not supported MySQL Mode* EER Diagram

Administration Schemas Sprint2 Leonardo Ruggiero* NI-Ex-1_datos_introducir NI-Ex-1_estructura datos NI-Ex-1_datos_introducir_credit

SCHEMAS

Filter objects

ejercicio4

Tables

Views

Stored Procedures

Functions

sys

transactions

Tables

company

credit_card

transaction

user

Views

Stored Procedures

Functions

1 -- Insertamos datos de credit_card

2 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-2938', 'TR30195031221357681636661', '5424465566813633', '3257', '984', '10/30/22');

3 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-2945', 'D026854763748537475216568689', '5142423821948828', '9800', '887', '08/24/23');

4 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-2952', 'B6451V0L52718525680255', '4556 453 55 5287', '4598', '438', '06/29/21');

5 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-2959', '07242477244335841535', '37246137749375', '3583', '667', '02/24/23');

6 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-2966', 'B072LX015627628377333', '448846 886747 7265', '4900', '130', '10/29/24');

7 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-2973', 'PT87866228135802429456346', '544 58654 54343 384', '8760', '887', '01/30/25');

8 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-2980', 'D63924188183806277136', '482408 7145845969', '5875', '596', '07/24/22');

9 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-2987', 'G689681434837748781813', '3763 747687 76666', '2290', '797', '10/31/23');

10 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-2994', 'B8657214428368866765294', '344283273252593', '7545', '595', '02/28/22');

11 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3001', 'CY49087426654774581266832110', '511722 924833 2244', '9562', '867', '09/16/22');

12 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3008', 'LU5072166936161192238', '4485744464433884', '1856', '740', '04/05/25');

13 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3015', 'PS119398216295715968342456821', '3784 662233 17389', '3246', '822', '01/31/22');

14 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3022', 'GT9169516285856977423121857', '5164 1379 4842 3951', '5610', '342', '04/25/25');

15 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3029', 'AZ62317143982441418123739746', '3429 279566 77631', '9708', '589', '09/02/23');

16 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3036', 'AZ39336082925842865843941994', '3768 451556 48766', '2232', '565', '10/27/25');

17 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3043', 'TM648814318514852179535', '455876 6457483635', '5969', '198', '06/07/25');

18 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3050', 'PS167744369175838331554477', '4824087122722', '4834', '126', '10/09/23');

19 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3057', 'U0931822574697545315', '3484 621767 21237', '6805', '848', '09/14/25');

20 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3064', 'PS14696554544925377627273133', '3467 732741 26818', '3865', '408', '06/03/25');

21 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3071', 'N08023814763512', '3464 789562 23352', '6625', '661', '12/20/23');

22 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3078', 'TS825127145884623279548733', '4539 322 74 2377', '9405', '720', '03/08/23');

23 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3085', 'BE63114723972437', '5266 3346 1135 1687', '7241', '413', '05/10/23');

24 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3092', 'R065L5001166122125447487', '3488 754223 46253', '9417', '594', '12/19/22');

25 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3099', 'PT26185273556823785537218', '448 55418 98863 789', '5612', '564', '01/22/23');

26 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3106', 'AT6845163775136592', '349547146395283', '9733', '289', '01/27/24');

27 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3113', 'IE26L6747732173572752', '34184822877471', '9011', '287', '06/12/21');

28 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3120', 'RS7265576666166237144', '527646 53375 6577', '7658', '265', '01/16/21');

29 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3127', 'PT8353461438644342816864', '4716 443 46 4368', '8038', '924', '01/16/23');

30 INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-3134', 'B623MY052668951824779', '5146 3453 9766 2168', '7260', '935', '08/24/25');

100%

Action Output

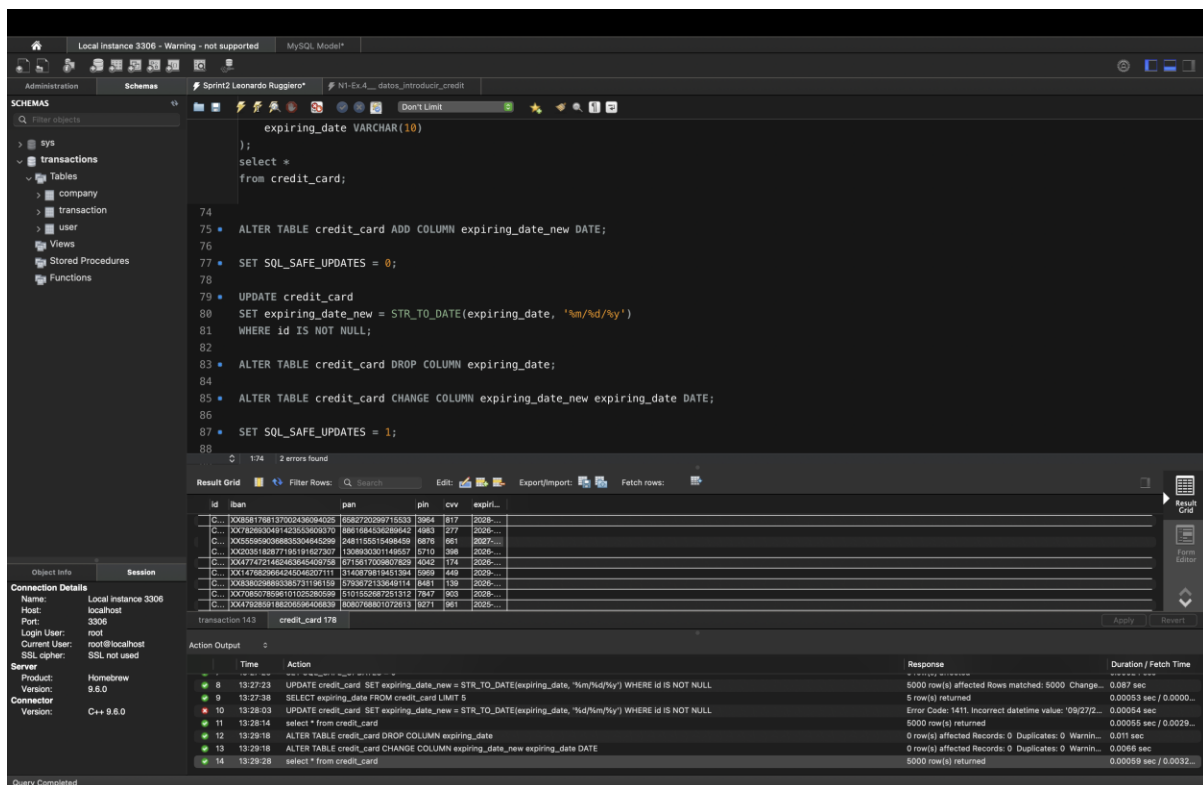
Time	Action	Response	Duration / Fetch Time
48... 19:40:14	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-9577', 'XX405448186289720388404508', '829682284225053', '1985', '220', '07/27/29');	1 row(s) affected	0.00016 sec
49... 19:40:14	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-9578', 'XX965338931053088901924906', '9099040779637171', '2277', '224', '01/25/28');	1 row(s) affected	0.00014 sec
49... 19:40:14	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-9574', 'XX06276758361432688520775', '37178331318656', '5084', '708', '11/28/27');	1 row(s) affected	0.00029 sec
49... 19:40:14	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-9575', 'XX4998812160726267196473', '8086033007377786', '7266', '960', '10/25/29');	1 row(s) affected	0.00015 sec
49... 19:40:14	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-9576', 'XX329702056577264772007', '4554223878064007', '4884', '724', '06/28/29');	1 row(s) affected	0.00016 sec
50... 19:40:14	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-9577', 'XX15891407898408633147086', '810437271856107', '5864', '772', '10/29/26');	1 row(s) affected	0.00015 sec
50... 19:40:14	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-9578', 'XX91953964646110567870254', '899808823081411', '2872', '772', '07/28/27');	1 row(s) affected	0.00015 sec
50... 19:40:14	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-9579', 'XX29639309158770202131236', '96900060468678689', '8378', '134', '12/25/25');	1 row(s) affected	0.00014 sec
50... 19:40:14	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-9580', 'XX78126888989195086677368', '944182384488931', '3273', '737', '03/27/29');	1 row(s) affected	0.00015 sec
50... 19:40:14	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CCU-9581', 'XX8158705164053812439847', '5824385470167839', '4336', '926', '06/28/25');	1 row(s) affected	0.00015 sec

Query Completed

RESPUESTA:

Para resolver el ejercicio, primero preparé la estructura en mi base de datos para recibir la información de los productos. Lo hice creando la tabla **products** con el comando CREATE TABLE, donde definí columnas como el ID, nombre, precio y color. Me aseguré de poner el **ID como llave primaria** para que cada producto sea único y no se duplique nada.

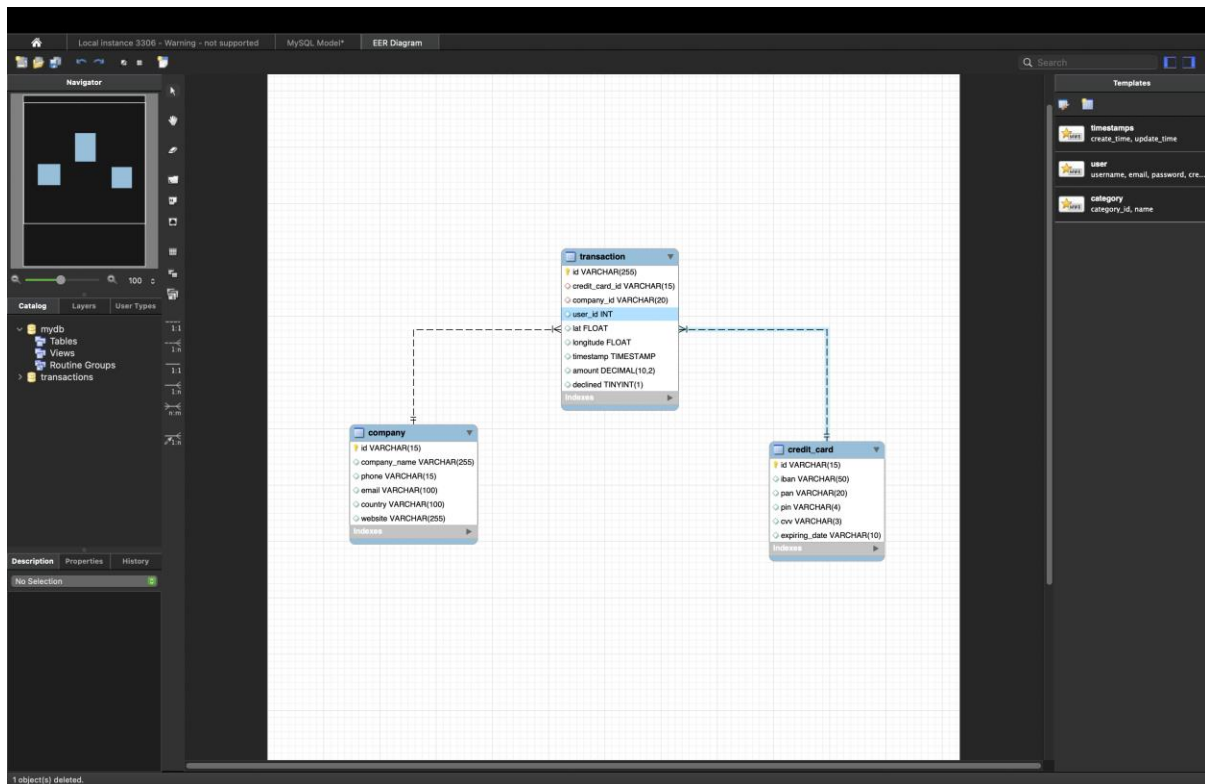
Después, para no escribir todo a mano, cargué los datos usando un **script de comandos INSERT INTO**. Simplemente abrí el archivo del script en mi editor de MySQL y lo ejecuté por completo. Esto llenó la tabla automáticamente con todos los registros, dejando todo listo para cruzar esa información con mis transacciones.



RESPUESTA:

Usé el comando STR_TO_DATE para transformar esos textos en fechas válidas. Sin embargo, me encontré con una traba de seguridad del **MySQL Workbench**: el sistema no me dejaba actualizar muchas filas a la vez para protegerme de errores accidentales. Por eso, tuve que usar el comando **SET SQL_SAFE_UPDATES = 0** antes de empezar. Sé que esto desactiva temporalmente esa protección, pero era la única forma de procesar todos los registros de una sola vez.

Después de ejecutar la actualización, volví a poner **SET SQL_SAFE_UPDATES = 1** para dejar el sistema seguro otra vez. Fue un paso necesario para que el formato cambiara correctamente sin tener que ir fila por fila.

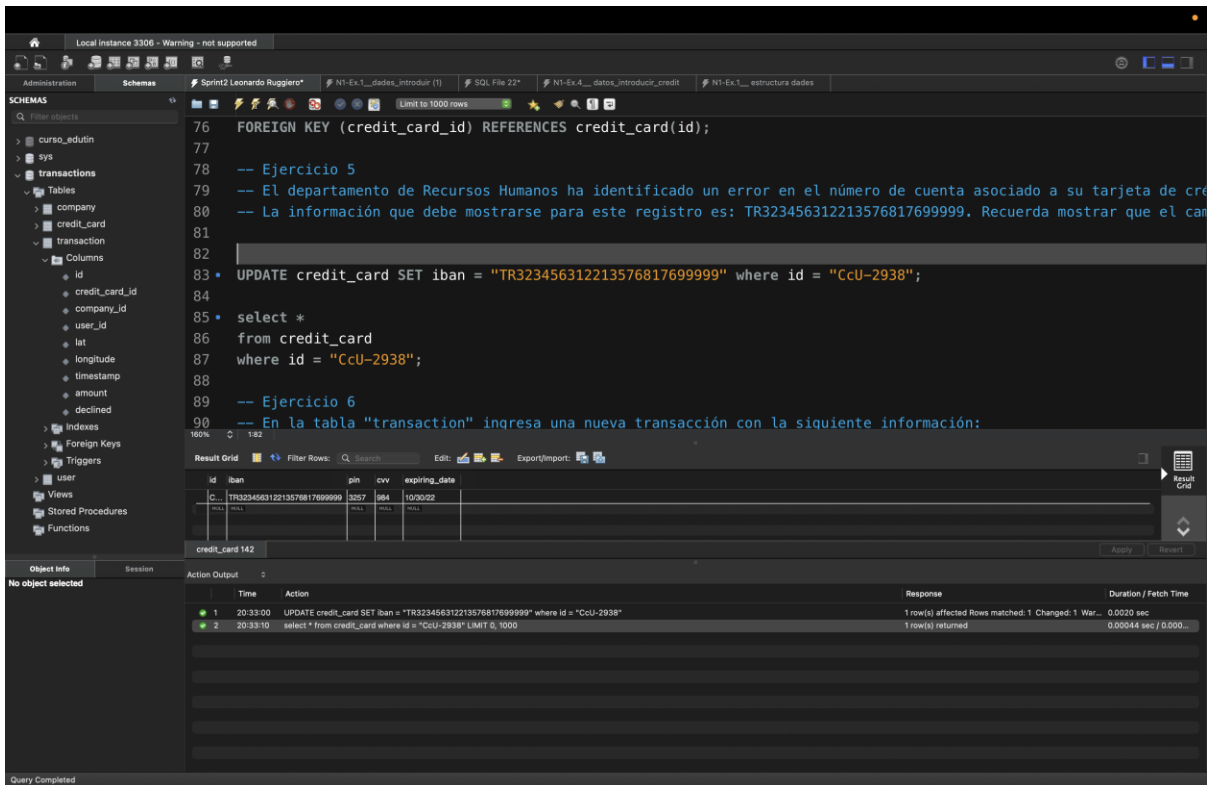


RESPUESTA:

Para visualizar cómo quedaron conectadas todas mis tablas, generé un diagrama de modelo relacional directamente en **MySQL Workbench**. No lo dibujé a mano, sino que utilicé la herramienta automática del programa siguiendo la ruta **Database > Reverse Engineer**.

Lo que hice fue pedirle al sistema que tomara las tablas que ya había creado y las transformara en un esquema visual. Al ejecutar este proceso, el Workbench analizó mis llaves primarias y relaciones para mostrarme el mapa completo de la base de datos. Es un paso que siempre hago para asegurarme de que la arquitectura de la información sea lógica y que todas las conexiones entre transacciones, productos y usuarios funcionen correctamente.

Ejercicio 5



Respuesta

Primero, utilicé el código "select * from credit_card where id = "CcU-2938";" para verificar los valores de la columna data. Luego, cambié el valor del IBAN que antes sería "TR301950312213576817638661" para "TR323456312213576817699999" usando "UPDATE credit_card SET iban = "TR323456312213576817699999" where id = "CcU-2938";" donde UPDATE modifica el valor existente al siguiente valor introducido en la línea de código. El "WHERE id" especifica qué PRIMARY KEY se modificará.

Ejercicio 6

The screenshot displays a SQL IDE interface. On the left, the 'SCHEMAS' pane shows a database structure with tables like 'transaction', 'company', 'credit_card', and 'user'. The central editor contains the following SQL code:

```
90
91
92
93
94 -- Ejercicio 6
95 -- En la tabla "transaction" ingresa una nueva transacción con la siguiente información:
96
97
98 INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, amount, declined)
99 VALUES ("108B1D1D-5B23-A76C-55EF-C568E49A99DD", "Ccu-9999", "b-9999", 9999, 829.999, -117.999, 111.11, 0);
100
101
102 select *
103 from transaction
104 where company_id = "b-9999";
105
106
107
```

Below the editor, the 'Results Grid' shows the result of the SELECT query:

id	credit_card...	company_id	user_id	lat	longitude	timestamp	amount	declined
108B1D1D-5B23-A76C-55EF-C568E49A99DD	Ccu-9999	b-9999	9999	829.999	-117.999	111.11	0	

The 'Action Output' pane at the bottom shows the execution log:

Time	Action	Response	Duration / Fetch Time
20:42:14	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, amount, declined) VALUES ("108B1D1D-5B23-A76C-55EF-C568E49A99DD", "Ccu-9999", "b-9999", 9999, 829.999, -117.999, 111.11, 0);	1 row(s) affected	0.00092 sec
20:42:31	select * from transaction where company_id = "b-9999" LIMIT 0, 1000	1 row(s) returned	0.00063 sec / 0.000...

Respuesta:

En este ejercicio, utilicé las funciones "INSERT INTO" y "VALUES" para agregar valores a la nueva columna solicitada. Sin embargo, tuve que crear nuevos campos USER, COMPANY y CREDIT_CARD porque los valores solicitados no existían en la tabla, lo que invalidaba mi función de inserción. Después de crear estos nuevos valores, intenté insertar la nueva columna nuevamente y la inserción fue exitosa.

Ejercicio 7

The screenshot shows a database management tool interface. On the left, a sidebar displays a tree view of database objects, including 'schemas', 'tables', and 'columns'. The 'credit_card' table is selected, and its columns are listed: id, iban, pan, pin, cvv, and expiring_date. The main area shows a SQL query editor with the following code:

```
-- Ejercicio 7
-- Desde recursos humanos te solicitan eliminar la columna "pan" de la tabla credit_card. Recuerda mostrar el cambio realizado.

130 select pan
131 from credit_card;
132
133 ALTER TABLE credit_card DROP column pan
```

Below the query editor, a 'Results Grid' displays the results of the query. The grid has columns for 'id', 'credit_card...', 'company_id', 'user_id', 'lat', 'longitude', 'timestamp', 'amount', and 'declined'. The first row shows 'null' for all columns. Below the grid, a 'transaction 143' summary is shown with 'Apply' and 'Revert' buttons.

At the bottom, an 'Action Output' panel shows the execution log:

Time	Action	Response	Duration / Fetch Time
21:44:35	select pan from credit_card LIMIT 0, 1000	1000 row(s) returned	0.0012 sec / 0.00011...
21:44:47	ALTER TABLE credit_card DROP column pan	0 row(s) affected Records: 0 Duplicates: 0 Warnin...	0.0098 sec

The status bar at the bottom indicates 'Query Completed'.

Respuesta:

Utilicé la función "ALTER TABLE" para modificar la tabla "credit_card" y "DROP COLUMN" para eliminar la columna "pan" de la tabla.

Ejercicio 8

Local Instance 3306 - Warning - not supported

Administration Schemas Sprint2 Leonardo Ruggiero* Don't Limit

SCHEMAS

Filter objects

sys transactions

Ejercicio 8

Descarga los archivos CSV que encontrarás en el apartado de recursos :

- american_users.csv
- european_users.csv
- companies.csv
- credit_cards.csv
- transactions.csv

Estudia y diseña una base de datos con un esquema de estrella que contenga, al menos 4 tablas de las que puedas realizar las siguientes consultas:

```
CREATE DATABASE IF NOT EXISTS ejercicio4;
USE ejercicio4;

CREATE TABLE user (
  id INT PRIMARY KEY,
  name VARCHAR(100),
  surname VARCHAR(100),
  phone VARCHAR(50),
  email VARCHAR(150),
  birth_date VARCHAR(50),
  country VARCHAR(100),
  city VARCHAR(100),
  postal_code VARCHAR(20),
  address VARCHAR(255)
);

CREATE TABLE company (
  id VARCHAR(15) PRIMARY KEY,
  company_name VARCHAR(255),
  phone VARCHAR(50),
  email VARCHAR(100),
  country VARCHAR(100),
  city VARCHAR(100),
  postal_code VARCHAR(20),
  website VARCHAR(255)
);

CREATE TABLE credit_card (
  id VARCHAR(15) PRIMARY KEY,
  user_id INT,
  iban VARCHAR(50),
  pan VARCHAR(20),
  pin VARCHAR(10),
  cvv VARCHAR(10),
  track1 VARCHAR(255),
  track2 VARCHAR(255),
  expiring_date DATE
);

CREATE TABLE transaction (
  id VARCHAR(255) PRIMARY KEY,
  card_id VARCHAR(15),
  business_id VARCHAR(15),
  timestamp TIMESTAMP,
  amount DECIMAL(10,2),
  declined TINYINT(1),
  product_id VARCHAR(255),
  user_id INT,
  lat FLOAT,
  longitude FLOAT,
  FOREIGN KEY (card_id) REFERENCES credit_card(id)
);
```

100% 30/183 1 error found

Action Output

	Time	Action	Response	Duration / Fetch Time
1	22:43:43	SHOW VARIABLES LIKE 'secure_file_priv'	1 row(s) returned	0.0046 sec / 0.00001...
2	23:22:20	SHOW VARIABLES LIKE 'secure_file_priv'	1 row(s) returned	0.0045 sec / 0.00000...
3	13:03:11	CREATE DATABASE IF NOT EXISTS ejercicio4	1 row(s) affected	0.0036 sec
4	13:03:11	USE ejercicio4	0 row(s) affected	0.00043 sec
5	13:03:11	CREATE TABLE user (id INT PRIMARY KEY, name VARCHAR(100), surname VARCHAR(100), phone VARCHAR(50), email VARCHAR(150), birth_date VARCHAR(50), country VARCHAR(100), city VARCHAR(100), postal_code VARCHAR(20), address VARCHAR(255))	0 row(s) affected	0.0076 sec
6	13:03:11	CREATE TABLE company (id VARCHAR(15) PRIMARY KEY, company_name VARCHAR(255), phone VARCHAR(50), email VARCHAR(100), country VARCHAR(100), city VARCHAR(100), postal_code VARCHAR(20), website VARCHAR(255))	0 row(s) affected	0.0044 sec
7	13:03:11	CREATE TABLE credit_card (id VARCHAR(15) PRIMARY KEY, user_id INT, iban VARCHAR(50), pan VARCHAR(20), pin VARCHAR(10), cvv VARCHAR(10), track1 VARCHAR(255), track2 VARCHAR(255), expiring_date DATE)	0 row(s) affected	0.0038 sec
8	13:03:11	CREATE TABLE transaction (id VARCHAR(255) PRIMARY KEY, card_id VARCHAR(15), business_id VARCHAR(15), timestamp TIMESTAMP, amount DECIMAL(10,2), declined TINYINT(1), product_id VARCHAR(255), user_id INT, lat FLOAT, longitude FLOAT, FOREIGN KEY (card_id) REFERENCES credit_card(id))	0 row(s) affected, 1 warning(s): 1681 Integer display...	0.0096 sec

Query Completed

Local Instance 3306 - Warning - not supported

Administration Schemas Sprint2 Leonardo Ruggiero* Don't Limit

SCHEMAS

Filter objects

sys transactions

```
company_name VARCHAR(255),
phone VARCHAR(50),
email VARCHAR(100),
country VARCHAR(100),
website VARCHAR(255)
);

CREATE TABLE credit_card (
  id VARCHAR(15) PRIMARY KEY,
  user_id INT,
  iban VARCHAR(50),
  pan VARCHAR(20),
  pin VARCHAR(10),
  cvv VARCHAR(10),
  track1 VARCHAR(255),
  track2 VARCHAR(255),
  expiring_date DATE
);

CREATE TABLE transaction (
  id VARCHAR(255) PRIMARY KEY,
  card_id VARCHAR(15),
  business_id VARCHAR(15),
  timestamp TIMESTAMP,
  amount DECIMAL(10,2),
  declined TINYINT(1),
  product_id VARCHAR(255),
  user_id INT,
  lat FLOAT,
  longitude FLOAT,
  FOREIGN KEY (card_id) REFERENCES credit_card(id)
);
```

100% 30/183 1 error found

Action Output

	Time	Action	Response	Duration / Fetch Time
1	22:43:43	SHOW VARIABLES LIKE 'secure_file_priv'	1 row(s) returned	0.0046 sec / 0.00001...
2	23:22:20	SHOW VARIABLES LIKE 'secure_file_priv'	1 row(s) returned	0.0045 sec / 0.00000...
3	13:03:11	CREATE DATABASE IF NOT EXISTS ejercicio4	1 row(s) affected	0.0036 sec
4	13:03:11	USE ejercicio4	0 row(s) affected	0.00043 sec
5	13:03:11	CREATE TABLE user (id INT PRIMARY KEY, name VARCHAR(100), surname VARCHAR(100), phone VARCHAR(50), email VARCHAR(150), birth_date VARCHAR(50), country VARCHAR(100), city VARCHAR(100), postal_code VARCHAR(20), address VARCHAR(255))	0 row(s) affected	0.0076 sec
6	13:03:11	CREATE TABLE company (id VARCHAR(15) PRIMARY KEY, company_name VARCHAR(255), phone VARCHAR(50), email VARCHAR(100), country VARCHAR(100), city VARCHAR(100), postal_code VARCHAR(20), website VARCHAR(255))	0 row(s) affected	0.0044 sec
7	13:03:11	CREATE TABLE credit_card (id VARCHAR(15) PRIMARY KEY, user_id INT, iban VARCHAR(50), pan VARCHAR(20), pin VARCHAR(10), cvv VARCHAR(10), track1 VARCHAR(255), track2 VARCHAR(255), expiring_date DATE)	0 row(s) affected	0.0038 sec
8	13:03:11	CREATE TABLE transaction (id VARCHAR(255) PRIMARY KEY, card_id VARCHAR(15), business_id VARCHAR(15), timestamp TIMESTAMP, amount DECIMAL(10,2), declined TINYINT(1), product_id VARCHAR(255), user_id INT, lat FLOAT, longitude FLOAT, FOREIGN KEY (card_id) REFERENCES credit_card(id))	0 row(s) affected, 1 warning(s): 1681 Integer display...	0.0096 sec

Query Completed

Local instance 3306 - Warning - not supported

Administration Schemas Sprint2 Leonardo Ruggiero*

Schemas

Filter objects

sys transactions

```
182 declen TINYINT(1),
183 product_ids VARCHAR(255),
184 user_id INT,
185 lat FLOAT,
186 longitude FLOAT,
187 FOREIGN KEY (card_id) REFERENCES credit_card(id),
188 FOREIGN KEY (business_id) REFERENCES company(id),
189 FOREIGN KEY (user_id) REFERENCES user(id)
190 };
191
192
193 SET FOREIGN_KEY_CHECKS = 0;
194
195 LOAD DATA INFILE '/Users/leonardoruggiero/mysql_imports/N1.Ex.8__ credit_cards.csv'
196 INTO TABLE credit_card
197 FIELDS TERMINATED BY ','
198 LINES TERMINATED BY '\n'
199 IGNORE 1 ROWS
200 (id, user_id, iban, pan, pin, cvv, track1, track2, @v_expiring_date)
201 SET expiring_date = STR_TO_DATE(@v_expiring_date, '%m/%d/%y');
202
203
204 LOAD DATA INFILE '/Users/leonardoruggiero/mysql_imports/N1.Ex.8__ transactions.csv'
205 INTO TABLE transaction
206 FIELDS TERMINATED BY ','
207 LINES TERMINATED BY '\n'
208 IGNORE 1 ROWS;
209
210
211 SET FOREIGN_KEY_CHECKS = 1;
```

100% 30:183 1 error found

Action Output

	Time	Action	Response	Duration / Fetch Time
1	22:43:43	SHOW VARIABLES LIKE 'secure_file_priv'	1 row(s) returned	0.0046 sec / 0.00001...
2	23:22:20	SHOW VARIABLES LIKE 'secure_file_priv'	1 row(s) returned	0.0046 sec / 0.00001...
3	13:03:11	CREATE DATABASE IF NOT EXISTS ejercicio4	1 row(s) affected	0.0036 sec
4	13:03:11	USE ejercicio4	0 row(s) affected	0.0043 sec
5	13:03:11	CREATE TABLE user (id INT PRIMARY KEY, name VARCHAR(100), surname VARCHAR(100), phone VARCHAR(50), email VARCHAR(150), birth_date VARCHAR(10), country VARCHAR(10), lat FLOAT, longitude FLOAT, FOREIGN KEY (card_id) REFERENCES credit_card(id), FOREIGN KEY (business_id) REFERENCES company(id), FOREIGN KEY (user_id) REFERENCES user(id));	0 row(s) affected	0.0078 sec
6	13:03:11	CREATE TABLE company (id VARCHAR(16) PRIMARY KEY, company_name VARCHAR(255), phone VARCHAR(50), email VARCHAR(100), country VARCHAR(10), lat FLOAT, longitude FLOAT, FOREIGN KEY (card_id) REFERENCES credit_card(id), FOREIGN KEY (business_id) REFERENCES company(id), FOREIGN KEY (user_id) REFERENCES user(id));	0 row(s) affected	0.0044 sec
7	13:03:11	CREATE TABLE credit_card (id VARCHAR(15) PRIMARY KEY, user_id INT, iban VARCHAR(50), pan VARCHAR(20), pin VARCHAR(10), cvv VARCHAR(10), track1 VARCHAR(16), track2 VARCHAR(16), expiring_date DATE, FOREIGN KEY (user_id) REFERENCES user(id), FOREIGN KEY (business_id) REFERENCES company(id), FOREIGN KEY (card_id) REFERENCES credit_card(id));	0 row(s) affected	0.0038 sec
8	13:03:11	CREATE TABLE transaction (id VARCHAR(255) PRIMARY KEY, card_id VARCHAR(15), business_id VARCHAR(15), timestamp TIMESTAMP, amount DECIMAL(10,2), FOREIGN KEY (card_id) REFERENCES credit_card(id), FOREIGN KEY (business_id) REFERENCES company(id), FOREIGN KEY (user_id) REFERENCES user(id));	0 row(s) affected, 1 warning(s): 1681 Integer display...	0.0096 sec

Query Completed

Local instance 3306 - Warning - not supported

Administration Schemas Sprint2 Leonardo Ruggiero*

Schemas

Filter objects

sys transactions

```
185 lat FLOAT,
186 longitude FLOAT,
187 FOREIGN KEY (card_id) REFERENCES credit_card(id),
188 FOREIGN KEY (business_id) REFERENCES company(id),
189 FOREIGN KEY (user_id) REFERENCES user(id)
190 );
191
192
193 SET FOREIGN_KEY_CHECKS = 0;
194
195 LOAD DATA INFILE '/tmp/N1-Ex.8__ american_users.csv'
196 INTO TABLE user FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\r\n' IGNORE 1 ROWS;
197
198 LOAD DATA INFILE '/tmp/N1-Ex.8__ european_users.csv'
199 INTO TABLE user FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\r\n' IGNORE 1 ROWS;
200
201
202 LOAD DATA INFILE '/tmp/N1-Ex.8__ companies.csv'
203 INTO TABLE company FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\r\n' IGNORE 1 ROWS;
204
205
206 LOAD DATA INFILE '/tmp/N1-Ex.8__ credit_cards.csv'
207 INTO TABLE credit_card
208 FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\n' IGNORE 1 ROWS
209 (id, user_id, iban, pan, pin, cvv, track1, track2, @v_expiring_date)
210 SET expiring_date = STR_TO_DATE(@v_expiring_date, '%m/%d/%y');
211
212
213 LOAD DATA INFILE '/tmp/N1-Ex.8__ transactions.csv'
214 INTO TABLE transaction
215 FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\n' IGNORE 1 ROWS;
216
217
218 SET FOREIGN_KEY_CHECKS = 1;
```

100% 78:199 1 error found

Action Output

	Time	Action	Response	Duration / Fetch Time
1	13:12:05	SET FOREIGN_KEY_CHECKS = 0	0 row(s) affected	0.00026 sec
2	13:12:05	LOAD DATA INFILE '/tmp/N1-Ex.8__ american_users.csv' INTO TABLE user FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\r\n' IGNORE 1 ROWS;	0 row(s) affected Records: 0 Deleted: 0 Skipped: 0	0.0025 sec
3	13:12:05	LOAD DATA INFILE '/tmp/N1-Ex.8__ european_users.csv' INTO TABLE user FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\r\n' IGNORE 1 ROWS;	0 row(s) affected Records: 0 Deleted: 0 Skipped: 0	0.0031 sec
4	13:12:05	LOAD DATA INFILE '/tmp/N1-Ex.8__ companies.csv' INTO TABLE company FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\r\n' IGNORE 1 ROWS;	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0	0.0029 sec
5	13:12:05	LOAD DATA INFILE '/tmp/N1-Ex.8__ credit_cards.csv' INTO TABLE credit_card FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\n' IGNORE 1 ROWS;	5000 row(s) affected Records: 5000 Deleted: 0 Skipped: 0	0.019 sec
6	13:12:05	LOAD DATA INFILE '/tmp/N1-Ex.8__ transactions.csv' INTO TABLE transaction FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\n' IGNORE 1 ROWS;	100000 row(s) affected Records: 100000 Deleted: 0 Skipped: 0	1.875 sec
7	13:12:07	SET FOREIGN_KEY_CHECKS = 1	0 row(s) affected	0.00022 sec

Query Completed

Local instance 3306 - Warning - not supported

Administration Schemas Sprint2 Leonardo Ruggiero*

SCHMAS

Filter objects

sys

transactions

```
212 FIELDS TERMINATED BY ';' ENCLOSED BY '"' LINES TERMINATED BY '\n' IGNORE 1 ROWS;
213
214 SET FOREIGN_KEY_CHECKS = 1;
215
216
217 USE ejercicio4;
218
219
220 LOAD DATA INFILE '/tmp/N1-Ex.8__american_users.csv'
221 INTO TABLE user FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\n' IGNORE 1 ROWS;
222
223 LOAD DATA INFILE '/tmp/N1-Ex.8__european_users.csv'
224 INTO TABLE user FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\n' IGNORE 1 ROWS;
225
226 LOAD DATA INFILE '/tmp/N1-Ex.8__companies.csv'
227 INTO TABLE company FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\r\n' IGNORE 1 ROWS;
228
229 LOAD DATA INFILE '/tmp/N1-Ex.8__credit_cards.csv'
230 INTO TABLE credit_card FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\n' IGNORE 1 ROWS
231 (id, user_id, iban, pan, pin, cvv, track1, track2, @v_expiring_date)
232 SET expiring_date = STR_TO_DATE(@v_expiring_date, '%m/%d/%y');
233
234 LOAD DATA INFILE '/tmp/N1-Ex.8__transactions.csv'
235 IGNORE INTO TABLE transaction FIELDS TERMINATED BY ';' ENCLOSED BY '"' LINES TERMINATED BY '\n' IGNORE 1 ROWS;
236
237 SET FOREIGN_KEY_CHECKS = 1;
238
239
240
241
```

100% 1/238 1 error found

Action Output

	Time	Action	Response	Duration / Fetch Time
1	13:24:13	LOAD DATA INFILE '/tmp/N1-Ex.8__american_users.csv' INTO TABLE user FIELDS TERMINATED BY ';' ENCLOSED BY '"' LINES TERMINATED BY '\n' IGNORE 1 ROWS	1010 row(s) affected Records: 1010 Deleted: 0 Skipped: 0	0.014 sec
2	13:24:13	LOAD DATA INFILE '/tmp/N1-Ex.8__european_users.csv' INTO TABLE user FIELDS TERMINATED BY ';' ENCLOSED BY '"' LINES TERMINATED BY '\n' IGNORE 1 ROWS	3990 row(s) affected Records: 3990 Deleted: 0 Skipped: 0	0.048 sec
3	13:24:13	LOAD DATA INFILE '/tmp/N1-Ex.8__companies.csv' INTO TABLE company FIELDS TERMINATED BY ';' ENCLOSED BY '"' LINES TERMINATED BY '\r\n' IGNORE 1 ROWS	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0	0.0022 sec
4	13:24:13	LOAD DATA INFILE '/tmp/N1-Ex.8__credit_cards.csv' INTO TABLE credit_card FIELDS TERMINATED BY ';' ENCLOSED BY '"' LINES TERMINATED BY '\n' IGNORE 1 ROWS	5000 row(s) affected Records: 5000 Deleted: 0 Skipped: 0	0.048 sec
5	13:24:13	LOAD DATA INFILE '/tmp/N1-Ex.8__transactions.csv' IGNORE INTO TABLE transaction FIELDS TERMINATED BY ';' ENCLOSED BY '"' LINES TERMINATED BY '\n' IGNORE 1 ROWS	100000 row(s) affected Records: 100000 Deleted: 0 Skipped: 0	2.104 sec
6	13:24:16	SET FOREIGN_KEY_CHECKS = 1	0 row(s) affected	0.00025 sec

Query Completed

Local instance 3306 - Warning - not supported

MySQL Model* EER Diagram

Administration Schemas Sprint2 Leonardo Ruggiero*

SCHMAS

Filter objects

ejercicio4

Tables

Views

Stored Procedures

Functions

sys

transactions

company

credit_card

transaction

user

Views

Stored Procedures

Functions

```
199
200 LOAD DATA INFILE '/tmp/N1-Ex.8__european_users.csv'
201 INTO TABLE user FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\n' IGNORE 1 ROWS;
202
203 LOAD DATA INFILE '/tmp/N1-Ex.8__companies.csv'
204 INTO TABLE company FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\r\n' IGNORE 1 ROWS;
205
206 LOAD DATA INFILE '/tmp/N1-Ex.8__credit_cards.csv'
207 INTO TABLE credit_card FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMINATED BY '\n' IGNORE 1 ROWS
208 (id, user_id, iban, pan, pin, cvv, track1, track2, @v_expiring_date)
209 SET expiring_date = STR_TO_DATE(@v_expiring_date, '%m/%d/%y');
210
211 LOAD DATA INFILE '/tmp/N1-Ex.8__transactions.csv'
212 IGNORE INTO TABLE transaction FIELDS TERMINATED BY ';' ENCLOSED BY '"' LINES TERMINATED BY '\n' IGNORE 1 ROWS;
213
214 SET FOREIGN_KEY_CHECKS = 1;
215
216
217 SELECT country, COUNT(*) as total
218 FROM user
219 GROUP BY country;
220
```

100% 18/219 1 error found

Result Grid

country	total
United States	520
United Kingdom	478
Canada	480
Spain	463
Netherlands	456
Sweden	417

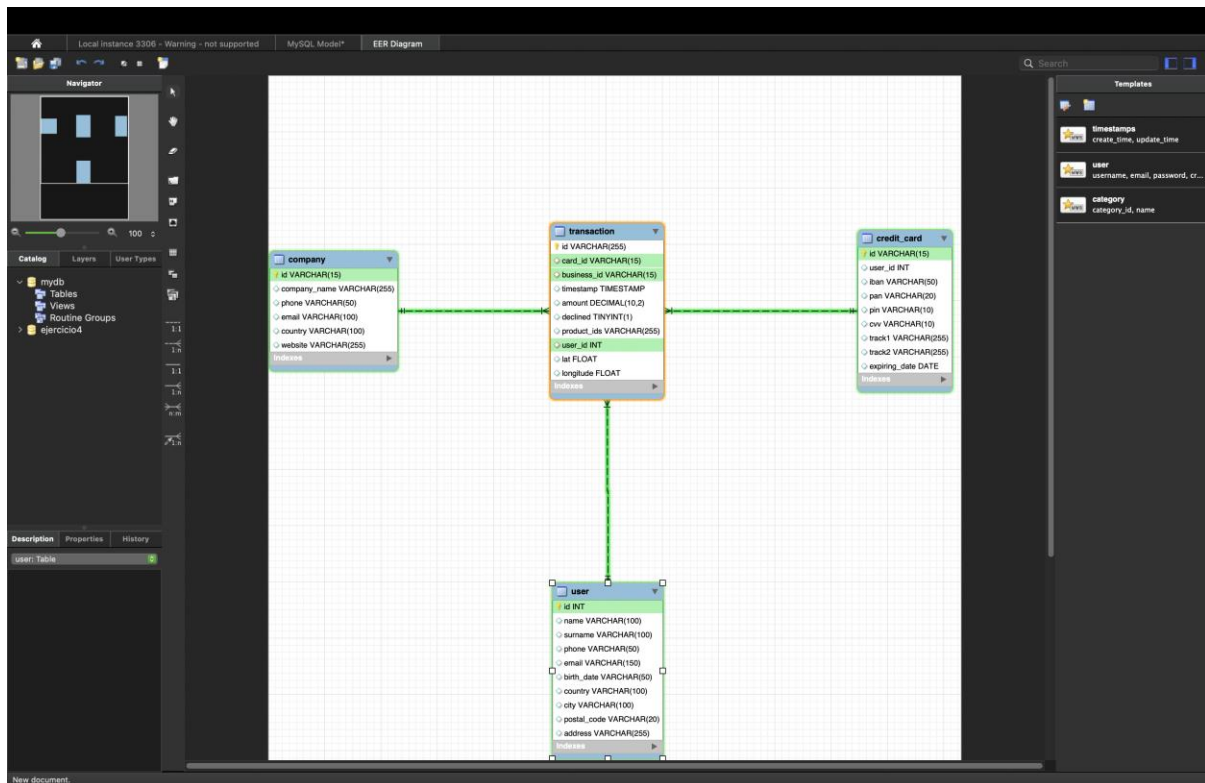
Result 5

Read Only

Action Output

	Time	Action	Response	Duration / Fetch Time
1	13:24:13	LOAD DATA INFILE '/tmp/N1-Ex.8__american_users.csv' INTO TABLE user FIELDS TERMINATED BY ';' ENCLOSED BY '"' LINES TERMINATED BY '\n' IGNORE 1 ROWS	1010 row(s) affected Records: 1010 Deleted: 0 Skipped: 0	0.014 sec
2	13:24:13	LOAD DATA INFILE '/tmp/N1-Ex.8__european_users.csv' INTO TABLE user FIELDS TERMINATED BY ';' ENCLOSED BY '"' LINES TERMINATED BY '\n' IGNORE 1 ROWS	3990 row(s) affected Records: 3990 Deleted: 0 Skipped: 0	0.048 sec
3	13:24:13	LOAD DATA INFILE '/tmp/N1-Ex.8__companies.csv' INTO TABLE company FIELDS TERMINATED BY ';' ENCLOSED BY '"' LINES TERMINATED BY '\r\n' IGNORE 1 ROWS	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0	0.0022 sec
4	13:24:13	LOAD DATA INFILE '/tmp/N1-Ex.8__credit_cards.csv' INTO TABLE credit_card FIELDS TERMINATED BY ';' ENCLOSED BY '"' LINES TERMINATED BY '\n' IGNORE 1 ROWS	5000 row(s) affected Records: 5000 Deleted: 0 Skipped: 0	0.048 sec
5	13:24:13	LOAD DATA INFILE '/tmp/N1-Ex.8__transactions.csv' IGNORE INTO TABLE transaction FIELDS TERMINATED BY ';' ENCLOSED BY '"' LINES TERMINATED BY '\n' IGNORE 1 ROWS	100000 row(s) affected Records: 100000 Deleted: 0 Skipped: 0	2.104 sec
6	13:24:16	SET FOREIGN_KEY_CHECKS = 1	0 row(s) affected	0.00025 sec
7	13:47:16	SELECT 'Usuarios' AS Tabla, COUNT(*) AS Total FROM user UNION ALL SELECT 'Compafias', COUNT(*) FROM company UNION ALL SELECT 'Tarjetas', COUNT(*) FROM credit_card	4 row(s) returned	0.024 sec / 0.000045...
8	13:48:16	SELECT c.company_name, COUNT(*) AS total_transacciones, SUM(t.amount) AS total_ventas FROM transaction t JOIN company c ON t.business_id = c.id GROUP BY c.id	10 row(s) returned	0.311 sec / 0.000010...
9	13:49:09	SELECT country, COUNT(*) as total FROM user GROUP BY country	11 row(s) returned	0.0045 sec / 0.0000...

Query Completed



Respuesta:

En primer lugar, analicé cada archivo CSV y registré sus respectivas columnas para proceder con la **creación de las tablas** user (unificando los archivos de american_users y european_users), company, credit_card y transaction. Diseñé cada estructura utilizando el formato de datos más óptimo y estableciendo **foreign keys** para vincular los registros de manera adecuada. (hice un pantallazo con un select para confirmar si la union de las tablas de users estaba correcta). Luego durante este proceso, identifiqué que la columna **expiring_date** no venía en formato de fecha, por lo que fue necesario transformarla utilizando la función **STR_TO_DATE** al momento de cargar la tabla credit_card. Para la integración de los datos, utilicé una línea de código 'INSERT' para cargar los archivos CSV, permitiendo que el SQL los procese e inserte en las tablas creadas. Finalmente, apliqué la instrucción **IGNORE** en la tabla transaction para evitar la duplicidad de registros y asegurar la integridad del esquema.

Ejercicio 9

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'ejercicio4' selected, containing 'Tables', 'Views', 'Stored Procedures', 'Functions', 'sys', and 'transactions'. The main editor shows a SQL query for 'Ejercicio 9' with the following code:

```
263
264 -- Ejercicio 9
265 -- Realiza una subconsulta que muestre a todos los usuarios con más de 80 transacciones utilizando al menos 2 tablas.
266
267
268 • SELECT id, name, surname, country
269 FROM user as u
270 WHERE EXISTS (
271     SELECT user_id
272     FROM transaction as t
273     WHERE t.user_id = u.id and declined = 0
274     GROUP BY t.user_id
275     HAVING COUNT(*) > 80);
276
277
278
```

Below the query editor, the 'Result Grid' shows the results of the query. The columns are 'id', 'name', 'surname', and 'country'. The results are as follows:

id	name	surname	country
186	Molly	Gilliam	United Kingdom
289	Oliver	Heene	Germany
318	Smyr	Astue	Italy

At the bottom, the 'Action Output' tab shows the execution details of the query, including the time taken and the number of rows returned.

RESPUESTA:

Primero hice un "SELECT" del "id" de la tabla de "user" seleccionando también "name", "surname" y "country" para que el resultado sea más visible, relacionando el "id" con la subconsulta que busca el "user_id" de la tabla "transaction", aplicando las funciones "GROUP BY" y "HAVING COUNT (*) > 80". De esta manera, el código realiza el filtro en la subconsulta según lo solicitado y lo compara con el "SELECT" de la consulta principal. He añadido "WHERE declined = 0" para que en la salida solo aparezcan transacciones aprobadas.

Ejercicio 10

```
219 GROUP BY country;
220
221 -- Ejercicio 9
222 -- Realiza una subconsulta que muestre a todos los usuarios con más de 80 transacciones utilizando al menos 2 tablas.
223
224
225 * SELECT id, name, surname, country
226 FROM user
227 WHERE id IN (SELECT user_id FROM transaction GROUP BY user_id HAVING COUNT(*) > 80);
228
229
230 -- Ejercicio 10
231 -- Muestra la media de amount por IBAN de las tarjetas de crédito en la compañía Donec Ltd., utiliza por lo menos 2 tablas.
232
233 * select * from company where company_name = "Donec Ltd";
234
235 * SELECT AVG(AMOUNT) as media_amount, iban, business_id
236 from transaction
237 join credit_card ON transaction.card_id = credit_card.id
238 where business_id = "b-2242"
239 group by iban;
240
```

id	company_name	phone	email	country	website
b-2242	Donec Ltd	01 25 51 37 37	alaculis@hotmail.co.uk	Norway	https://nytimes.com/user/110

company 34

Time	Action	Response	Duration / Fetch Time
15:15:49	select * from company where company_name = "Donec Ltd"	1 row(s) returned	0.00073 sec / 0.0000...

Query Completed

```
214 * SET FOREIGN_KEY_CHECKS = 1;
215
216
217 * SELECT country, COUNT(*) as total
218 FROM user
219 GROUP BY country;
220
221
222 -- Ejercicio 9
223 -- Realiza una subconsulta que muestre a todos los usuarios con más de 80 transacciones utilizando al menos 2 tablas.
224
225 * SELECT id, name, surname, country
226 FROM user
227 WHERE id IN (SELECT user_id FROM transaction GROUP BY user_id HAVING COUNT(*) > 80);
228
229
230 -- Ejercicio 10
231 -- Muestra la media de amount por IBAN de las tarjetas de crédito en la compañía Donec Ltd., utiliza por lo menos 2 tablas.
232
233 * select * from company where company_name = "Donec Ltd";
234
235 * SELECT AVG( ) as
```

media_amount	iban	business_id
358.246667	XX811406401125586307586805	b-2242
143.980000	SR9446370524745262577808	b-2242
157.370000	XX41972520178450626270509640	b-2242
139.590000	XX413827952289719304928990	b-2242
140.410000	XX347787246070769610760308	b-2242
158.580000	XX680784535432006484545002	b-2242

Result 35

Time	Action	Response	Duration / Fetch Time
15:15:49	select * from company where company_name = "Donec Ltd"	1 row(s) returned	0.00073 sec / 0.0000...
15:16:25	SELECT AVG(AMOUNT) as media_amount, iban, business_id from transaction join credit_card ON transaction.card_id = credit_card.id where business_id = "b-2242" gro...	371 row(s) returned	0.0056 sec / 0.0000...

Query Completed

RESPUESTA:

En este ejercicio, comencé obteniendo el 'id' de la compañía 'Donec Ltd.' haciendo una primera consulta para no tener que hacer un 'JOIN' con la tabla 'company'.

Después de obtener el 'id' de la empresa, empecé la construcción del ejercicio haciendo una selección del 'avg' (promedio) de 'amount', seleccionando el 'iban' y el 'business_id' para filtrar la respuesta. Luego, realicé un 'JOIN' uniendo las tablas 'credit_card' y 'transaction' por las claves primarias para saber cuántos ibans y valores de transacciones tendría el “b-2242” (el id de Donec Ltd.) usando ‘where = B-2242' y ‘where = declined = 0’ para eliminar transacciones nulas, finalizando con el 'group by' agrupando por el iban."

Nivel 2

Ejercicio 1

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'ejercicio4' selected, showing tables like 'company', 'credit_card', 'transaction', and 'user'. The main editor contains the following SQL query:

```

238 where business_id = "b-2242"
239 group by iban;
240
241
242
243 -- Nivel 2
244
245 -- Ejercicio 1
246 -- Identifica los cinco días que se generó la mayor cantidad de ingresos en la empresa por ventas. Muestra la fecha de cada transacción junto con el total de las ventas.
247
248
249 select DATE(timestamp) as dias, sum(amount) as mayor_cantidad_de_ingresos, company_name
250 from transaction
251 join company
252 on transaction.business_id = company.id
253 where declined = 0
254 GROUP BY dias, company.company_name
255 ORDER BY mayor_cantidad_de_ingresos desc
256 LIMIT 5;
257
258
259

```

The 'Result Grid' shows the following data:

dias	mayor_cantidad_de_ingre...	company_name
2024-12-02	3398.40	Eget Ipsum Ltd
2019-03-19	3149.04	Ac Fermentum Incorporated
2017-12-20	2774.04	Nulla Integer Vulputate Corp.
2017-12-20	2273.35	Viverra Donec Egetidion
2020-12-11	2717.76	Ac Fermentum Incorporated

The 'Action Output' pane at the bottom shows the execution log with 13 steps, including the final query execution which returned 5 rows.

RESPUESTA:

Comencé filtrando los valores solicitados en el enunciado, que son: la fecha (utilicé **DATE(timestamp)**), el **SUM(amount)** para calcular la suma de las transacciones y el nombre de la empresa, que entra en los filtros seleccionados uniendo la tabla

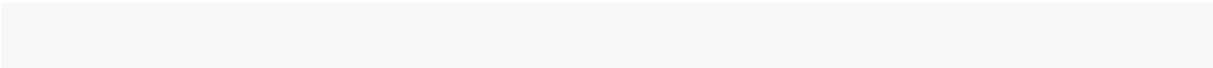
transaction con **company** para obtener el nombre de las empresas. Tras esta construcción, solo faltaron las funciones:

Transacciones aprobadas (declined = 0).

Agrupacion por días y nombre de las empresas (GROUP BY días, company_name).

Las cinco primeras del tope (ORDER BY mayor_cantidad_de_ingresos DESC, LIMIT 5).

Y así obtener el resultado pedido en el ejercicio.



Ejercicio 2

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
307 -- Ejercicio 2
308 -- Presenta el nombre, teléfono, país, fecha y amount, de aquellas empresas que realizaron transacciones con un valor comprendido
309 -- entre 350 y 400 euros y en alguna de estas fechas:
310 -- 29 de abril de 2015, 20 de julio de 2018 y 13 de marzo de 2024. Ordena los resultados de mayor a menor cantidad.
311
312 SELECT company_name, phone, country, DATE(timestamp) as fecha, amount, declined
313 FROM transaction as t
314 JOIN company as c
315 ON t.business_id = c.id
316 WHERE t.amount BETWEEN 350 AND 400
317 AND (DATE(t.timestamp) = "2015-04-29" or DATE(t.timestamp) = "2018-07-20" or DATE(t.timestamp) = "2024-03-13")
318 AND t.declined = "0"
319 ORDER BY amount DESC;
```

The Results window shows the following data:

company_name	phone	country	fecha	amount	declined
Aliquam PC	01 43 73 52 16	Germany	2024-03-13	399.84	0
Auctor Mauris Vel LLP	08 59 28 74 14	United States	2018-07-20	386.51	0
Aliquam PC	01 43 73 52 16	Germany	2024-03-13	386.29	0
Aliquam PC	01 43 73 52 16	Germany	2024-03-13	386.29	0
Ono Advertising Limited	03 15 00 77 81	United Kingdom	2015-04-29	372.17	0
Fringilla LLC	08 29 15 83 57	New Zealand	2015-04-29	367.62	0
Pede Cum Ltd	07 62 28 48 38	Norway	2018-07-20	356.87	0
Auctor Mauris Vel LLP	08 59 28 74 14	United States	2024-03-13	353.75	0

The Action Output window shows the execution details:

Time	Action	Response	Duration / Fetch Time
59 13:36:33	SELECT company_name, phone, country, DATE(timestamp) as fecha, amount, declined FROM transaction as t JOIN company as c ON t.business_id = c.id WHERE t.amount...	8 row(s) returned	0.051 sec / 0.000012...
60 13:36:39	SELECT company_name, phone, country, DATE(timestamp) as fecha, amount, declined FROM transaction as t JOIN company as c ON t.business_id = c.id WHERE t.amount...	8 row(s) returned	0.051 sec / 0.000014...

RESPUESTA

"En este ejercicio, comencé identificando las tablas necesarias para conectar la información:

-**'transaction'** para los montos y fechas

- **'company'** para los nombres y datos de contacto.

-Realicé un 'JOIN' uniendo ambas tablas a través de la clave **'business_id'** de la TRANSACTION y el **'company_id'** de la COMPANY para poder visualizar el nombre de la compañía junto a sus ventas.

-Posteriormente, construí el filtro principal en la cláusula 'WHERE' utilizando el operador **'BETWEEN'** para delimitar los valores de 'AMOUNT' exclusivamente entre **350 y 400 euros**. Para la selección de las fechas, utilicé la función **'DATE()'** sobre el campo 'TIMESTAMP' para ignorar las horas y apliqué la condicional **'WHERE'**, lo que me permitió hacer la condicion con 'OR' para agregar la condicion del enunciado (FECHA = **29 de abril de 2015 or 20 de julio de 2018 or 13 de marzo de 2024**) en una sola condición lógica y agrego la condicion **'declined = 0'** evitando transacciones rechazadas.

-Finalmente, seleccioné las columnas solicitadas (company_name, phone, country, fecha y amount) y terminé la construcción aplicando un 'ORDER BY' sobre el monto de manera descendente, organizando así los resultados de mayor a menor cantidad.

Ejercicio 3

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'ejercicio4' selected, showing tables like 'company', 'transaction', and 'credit_card'. The main editor contains a SQL query:

```
206 on transaction.business_id = company.id
207 where amount BETWEEN 350 AND 400 and DATE(timestamp) IN ("2015-04-29", "2018-06-20", "2024-03-13")
208 order by amount desc;
209
210 -- Ejercicio 3
211 -- Necesitamos optimizar la asignación de los recursos y dependerá de la capacidad operativa que se requiera, por lo que te piden la información sobre la cantidad de transacciones
212 -- que realizan las empresas, pero el departamento de recursos humanos es exigente y quiere un listado de las empresas en las que especifiques si tienen igual o más de
213 -- 400 transacciones o menos.
214
215 SELECT c.company_name, COUNT(t.id) AS total_transacciones,
216 CASE
217 WHEN COUNT(t.id) >= 400 THEN 'Igual o más de 400'
218 ELSE 'Menos de 400'
219 END AS contaje_valores
220 FROM company c
221 JOIN transaction t ON t.business_id = c.id
222 GROUP BY c.company_name
223 ORDER BY total_transacciones;
```

The 'Result Grid' shows the following data:

company_name	total_transacciones	contaje_valores
Loxem Eu Incorporated	380	Menos de 400
Fringilla LLC	397	Menos de 400
Nec Luctus LLC	398	Menos de 400
Qui Quis Institute	402	Igual o más de 400
Ruam Associates	407	Igual o más de 400
Magna A Neque Industries	410	Igual o más de 400

The 'Action Output' pane at the bottom shows the execution of the query, including the time taken (0.00052 sec) and the number of rows returned (100 rows).

RESPUESTA:

Para cumplir con lo solicitado, yo hice una consulta vinculando **company** y **transaction** mediante un “**JOIN**”. Utilice “**COUNT**” y “**GROUP BY**” para totalizar las ventas por empresa. La lógica clave se con “**CASE WHEN**”, definiendo que si el conteo es igual o mayor a 400, se asigne el texto '**Igual o más de 400**', y mediante “**ELSE**”, se etiquete el resto como '**Menos de 400**'. El uso de comillas simples fue esencial para identificar los textos, logrando un reporte que categoriza la capacidad operativa para Recursos Humanos.

Ejercicio 4

The screenshot shows the MySQL Workbench interface with the following content:

SQL Editor:

```
269 -- Ejercicio 3
270 -- Necesitamos optimizar la asignación de los recursos y dependerá de la capacidad operativa que se requiera, por lo que te piden la información sobre la cantidad de transacciones
271 -- que realizan las empresas, pero el departamento de recursos humanos es exigente y quiere un listado de las empresas en las que especifiques si tienen igual o más de
272 -- 400 transacciones o menos.
273
274
275 SELECT c.company_name, COUNT(t.id) AS total_transacciones,
276 CASE
277   WHEN COUNT(t.id) >= 400 THEN 'Igual o más de 400'
278   ELSE 'Menos de 400'
279 END AS contaje_valores
280 FROM company c
281 JOIN transaction t ON t.business_id = c.id
282 GROUP BY c.company_name
283 ORDER BY total_transacciones;
284
285 -- Ejercicio 4
286 -- Elimina de la tabla transacción el registro con ID 000447FE-B650-4DCF-85DE-C7ED0EE1CAAD de la base de datos.
287
288
289 DELETE FROM transaction
290 where id = "000447FE-B650-4DCF-85DE-C7ED0EE1CAAD";
291
292
293
294
295
296
297
298
```

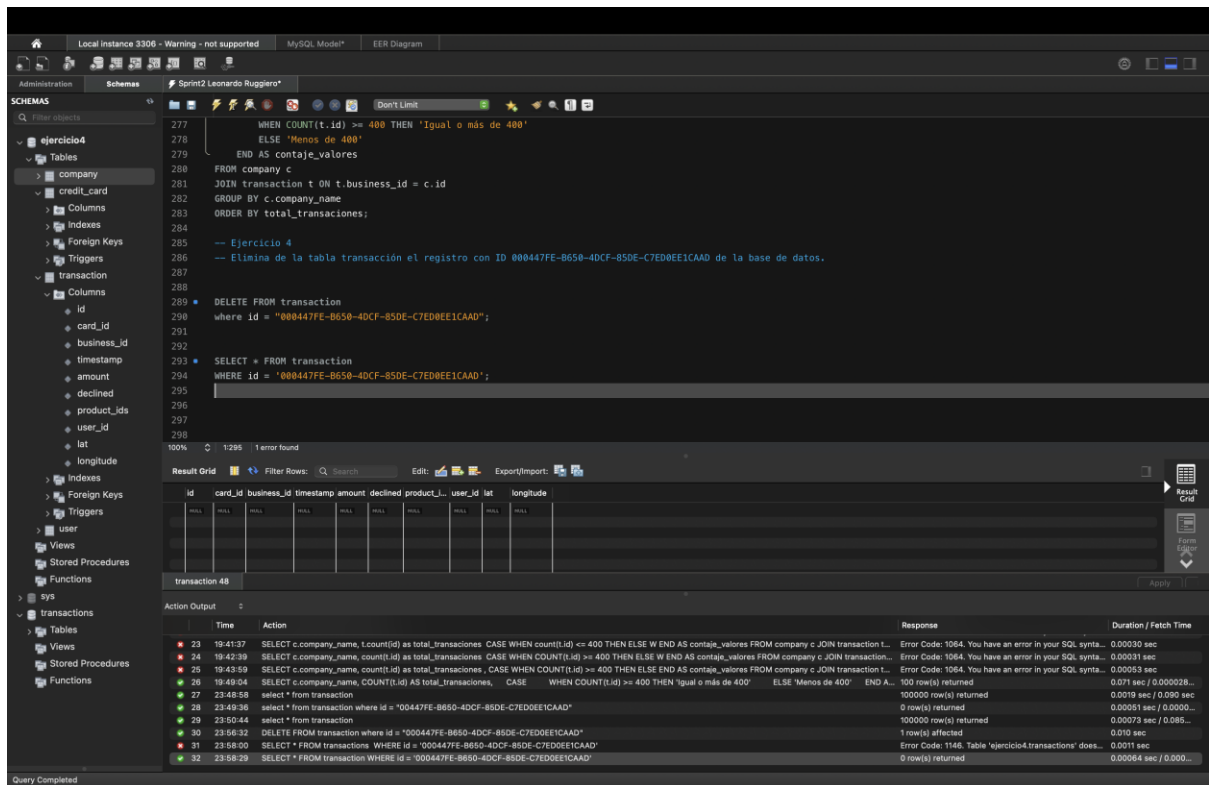
Action Output:

Time	Action	Response	Duration / Fetch Time
21 19:34:11	select count(id) from transaction	1 row(s) returned	0.026 sec / 0.00012...
22 19:34:11	case when id <= 400 then >= 400 end	Error Code: 1064. You have an error in your SQL synta...	0.00037 sec
23 19:41:37	SELECT c.company_name, t.count(id) as total_transacciones CASE WHEN count(t.id) <= 400 THEN ELSE W END AS contaje_valores FROM company c JOIN transaction t...	Error Code: 1064. You have an error in your SQL synta...	0.00030 sec
24 19:42:39	SELECT c.company_name, count(t.id) as total_transacciones CASE WHEN COUNT(t.id) >= 400 THEN ELSE W END AS contaje_valores FROM company c JOIN transaction...	Error Code: 1064. You have an error in your SQL synta...	0.00031 sec
25 19:43:59	SELECT c.company_name, count(t.id) as total_transacciones CASE WHEN COUNT(t.id) >= 400 THEN ELSE AS contaje_valores FROM company c JOIN transaction L...	Error Code: 1064. You have an error in your SQL synta...	0.00053 sec
26 19:49:04	SELECT c.company_name, COUNT(t.id) AS total_transacciones, CASE WHEN COUNT(t.id) >= 400 THEN 'Igual o más de 400' ELSE 'Menos de 400' END A...	100 row(s) returned	0.071 sec / 0.000028...
27 23:48:58	select * from transaction	100000 row(s) returned	0.0019 sec / 0.090 sec
28 23:49:36	select * from transaction where id = "00447FE-B650-4DCF-85DE-C7ED0EE1CAAD"	0 row(s) returned	0.00051 sec / 0.0000...
29 23:50:44	select * from transaction	100000 row(s) returned	0.00073 sec / 0.086...
30 23:56:32	DELETE FROM transaction where id = "000447FE-B650-4DCF-85DE-C7ED0EE1CAAD"	1 row(s) affected	0.010 sec

Query Completed

RESPUESTA:

En este ejercicio, utilicé la instrucción 'DELETE FROM transaction' junto con la cláusula 'WHERE' para localizar el ID solicitado '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD' y realizar la eliminación requerida en el enunciado.



RESPUESTA:

También realicé una captura de pantalla (printscreen) de una consulta 'SELECT' filtrando por ese mismo ID para confirmar que todo fue eliminado correctamente y que el registro ya no existe en la base de datos.

Ejercicio 5

Local instance 3306 - Warning - not supported MySQL Model* EER Diagram

Schemas Sprint2 Leonardo Ruggiero* new_view - View

293 SELECT * FROM transaction
294 WHERE id = '000447FE-B650-4DCF-850E-C7ED0EE1CA0D';
295
296
297 -- Ejercicio 5
298 -- La sección de marketing desea tener acceso a información específica para realizar análisis y estrategias efectivas. Se ha solicitado crear una vista que proporcione
299 -- detalles clave sobre las compañías y sus transacciones. Será necesaria que crees una vista llamada VistaMarketing que contenga la siguiente información: Nombre de la compañía.
300 -- Teléfono de contacto, País de residencia, Media de compra realizado por cada compañía. Presenta la vista creada, ordenando los datos de mayor a menor promedio de compra.
301
302 CREATE VIEW VistaMarketing AS
303 SELECT company_name, phone, country, AVG(amount) AS media_por_compania
304 FROM company
305 JOIN transaction ON company.id = transaction.business_id
306 WHERE declined = 0
307 group by company_name, phone, country;
308
309 SELECT * FROM VistaMarketing
310 ORDER BY media_por_compania DESC;
311
312
313
314

100% 1:311 1 error found

Result Grid Filter Rows: Search Export

company_name	phone	country	media_por_compania...
Ac Fermentum Incorporated	06 85 58 52 33	Germany	284.911333
Preibus Nunc Corp.	07 77 48 85 28	Australia	275.876041
Utra Convalis Associates	06 01 24 77 04	United States	272.568925
At Associates	09 58 61 10 85	New Zealand	272.736985
Metus Vitor Associates	08 28 44 40 58	Australia	270.251876
Aliquet Diam Limited	02 78 61 47 48	United States	269.291145

VistaMarketing 50

Action Output

Time	Action	Response	Duration / Fetch Time
34 10:23:41	SELECT company_name, company.id, phone, country, AVG(amount) AS media_por_compania FROM company JOIN transaction ON company.id = transaction.business_id...	Error Code: 1140. In aggregated query without GROU...	0.0024 sec
35 10:32:12	CREATE VIEW VistaMarketing AS SELECT company_name, company.id, phone, country, AVG(amount) AS media_por_compania FROM company JOIN transaction ON com...	0 row(s) affected	0.0081 sec
36 10:32:12	SELECT * FROM VistaMarketing ORDER BY media_por_compania DESC	Error Code: 1055. Expression #2 of SELECT list is not...	0.0012 sec
37 10:32:33	CREATE VIEW VistaMarketing AS SELECT company_name, company.id, phone, country, AVG(amount) AS media_por_compania FROM company JOIN transaction ON com...	Error Code: 1050. Table 'VistaMarketing' already exists	0.0012 sec
38 10:33:27	DROP VIEW VistaMarketing	0 row(s) affected	0.0027 sec
39 10:34:04	CREATE VIEW VistaMarketing AS SELECT company_name, company.id, phone, country, AVG(amount) AS media_por_compania FROM company JOIN transaction ON com...	0 row(s) affected	0.0024 sec
40 10:34:42	SELECT * FROM VistaMarketing ORDER BY media_por_compania DESC	Error Code: 1055. Expression #2 of SELECT list is not...	0.0012 sec
41 10:35:18	DROP VIEW VistaMarketing	0 row(s) affected	0.0018 sec
42 10:35:28	CREATE VIEW VistaMarketing AS SELECT company_name, phone, country, AVG(amount) AS media_por_compania FROM company JOIN transaction ON company.id = tran...	0 row(s) affected	0.0023 sec
43 10:35:34	SELECT * FROM VistaMarketing ORDER BY media_por_compania DESC	100 row(s) returned	0.272 sec / 0.000035...

Query Completed

RESPUESTA:

Primero construí la vista y la nombré como pide el ejercicio usando "**CREATE VIEW as (nombre de la vista)**", luego creé los "**SELECT**" junto con la media de las transacciones que se pide al principio del ejercicio. Realicé un "**JOIN**" uniendo la tabla "**company**" con la tabla "**transaction**" porque el "**amount**" del cual necesito obtener la media está en la tabla de transacción. Usé la condición "**WHERE declined = 0**" para devolver solo transacciones efectuadas y agrupé con "**GROUP BY**" por las columnas "**company_name**", "**phone**" y "**country**". Al final, hice el "**SELECT**" de toda la vista ordenando la media de mayor a menor con "**ORDER BY DESC**" para filtrar el resultado deseado.

Nivel 3

Ejercicio 1

The screenshot shows the MySQL Workbench interface. The top toolbar includes icons for file operations, database management, and query execution. The left sidebar displays the 'SCHEMAS' tree with 'ejercicio4' selected. The main editor contains a SQL query for 'Nivel 3 Ejercicio 1'. The query creates a view 'VistaMarketing' and then selects data from it, using window functions to determine card status based on recent transactions. The 'Result Grid' at the bottom shows the output of the query, with columns 'status' and 'total_tarjetas'. The 'Action Output' pane at the very bottom shows the execution log of the SQL statements.

```
WITH UltimasTransacciones AS
(SELECT card_id, declined, ROW_NUMBER() OVER (PARTITION BY card_id ORDER BY timestamp DESC) as fila
FROM transaction)

322 SELECT status, COUNT(*) AS total_tarjetas
323 FROM (SELECT card_id,
324 CASE WHEN SUM(declined) = COUNT(declined) AND COUNT(declined) >= 3 THEN 'Inactive'
325 ELSE 'Activo'
326 END AS status
327 FROM UltimasTransacciones
328 WHERE fila <= 3
329 GROUP BY card_id
330 ) AS ResumenEstado
331 GROUP BY status
332
```

status	total_tarjetas
Activo	4995
Inactivo	5

RESPUESTA:

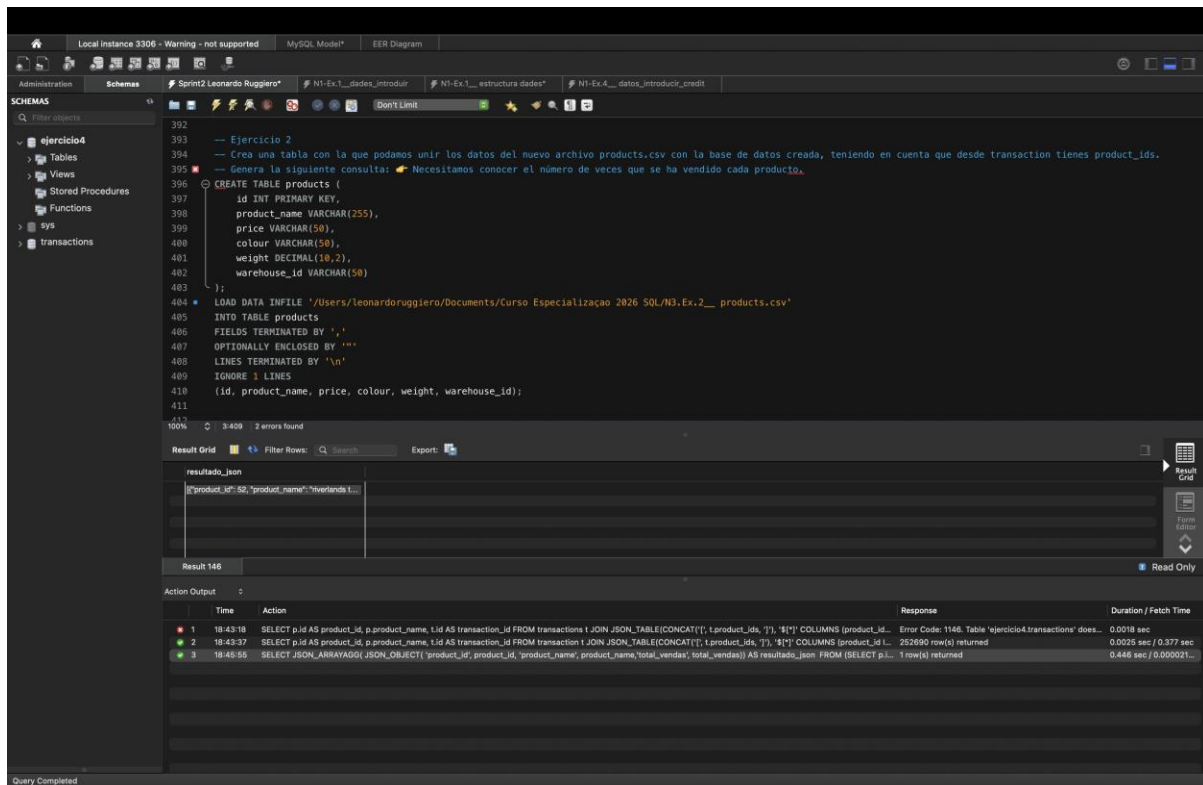
En este ejercicio, utilicé **Funciones de Ventana (Window Functions)** para evaluar el comportamiento reciente de cada tarjeta. El proceso se detalla a continuación:

Primero, creé una **Expresión de Tabla Común** llamada 'UltimasTransacciones'. Dentro de ella, utilicé la función '**ROW_NUMBER()**' combinada con '**OVER(PARTITION BY card_id ORDER BY timestamp DESC)**'. Esta instrucción es fundamental porque 'numera' las transacciones de cada tarjeta individualmente, asignando el número 1 a la más reciente y continuando en orden descendente.

Después, en la consulta principal, filtré los resultados para considerar únicamente las filas donde el número asignado fuera **menor o igual a 3**. Sobre este subconjunto de datos, apliqué una lógica de agregación: si la suma de las transacciones marcadas como rechazadas ('declined' = 1) es igual al total de transacciones analizadas para esa tarjeta, el sistema le asigna el estado de '**Inactivo**'. En cualquier otro caso (si hay al menos una transacción exitosa), la tarjeta se clasifica como '**Activo**'.

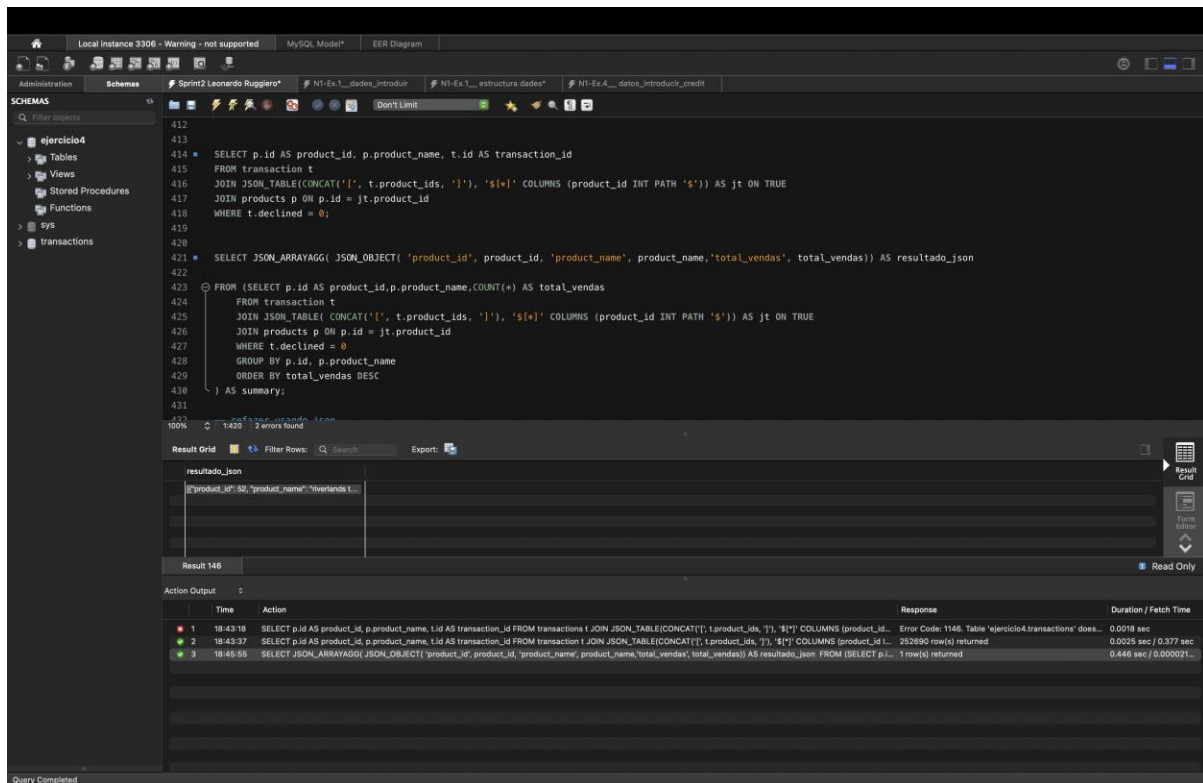
Finalmente, agrupé los resultados por el nuevo estado generado para obtener el conteo total. El análisis confirmó que existen **4.995 tarjetas activas**, ya que la gran mayoría presentó al menos un movimiento exitoso dentro de sus últimas tres operaciones

Ejercicio 2



RESPUESTA:

Primero usé el comando **CREATE TABLE** para fabricar el "molde" o la caja donde guardaría los productos. Definí qué columnas necesitaba (ID, nombre, precio, etc.) y le puse una **Primary Key** al ID. Hice esto para que cada producto sea único y no se mezclen los datos. Con la tabla ya creada, usé el comando **INSERT INTO** para meter la información que venía en el archivo CSV, simplemente "llené" los huecos. Me aseguré de que cada dato cayera en su columna correspondiente (el nombre en la columna de nombres, el precio en la de precios). Es el paso donde la tabla cobra vida con datos reales.



RESPUESTA:

Para resolver este ejercicio, mi prioridad fue transformar un campo desordenado (la lista de IDs) en datos que el motor de base de datos pudiera contar de forma eficiente. Este fue mi proceso: Me di cuenta de que product_ids guardaba los datos como texto (ej: "1, 2, 3"). Mi primera decisión fue **convertir esa cadena en un formato JSON real**. Para ello, usé CONCAT para añadir corchetes al principio y al final, transformando el texto en un array: [1, 2, 3].

Una vez que tuve el array, necesitaba que cada número ocupara su propia fila. Para esto usé **JSON_TABLE**. Es la función que elegí para "romper" el array y generar una tabla virtual donde cada ID de producto es un registro independiente. Sin esto, no podría haber hecho el cálculo.

Solo me interesaban las ventas reales, así que filtré por declined = 0. Después, hice un **JOIN** con mi tabla de productos. Hice esto para cruzar el ID que extraje del JSON con el nombre real del producto, asegurándome de que los datos fueran legibles y no solo números. Utilicé **JSON_OBJECT** para definir la estructura de cada par de datos (ID, nombre, total) y luego envolví todo con **JSON_ARRAYAGG**. Esto lo hice para que el resultado final sea un único bloque de datos listo para ser usado por cualquier aplicación.

Resumen de las funciones que apliqué:

CONCAT(): La usé para "engañar" al sistema y que viera la columna como una lista JSON válida.

JSON_TABLE(): Mi herramienta clave para convertir esa lista en filas de una tabla.

JSON_OBJECT(): La usé para darle nombre a las etiquetas dentro del resultado (como "total_vendas").

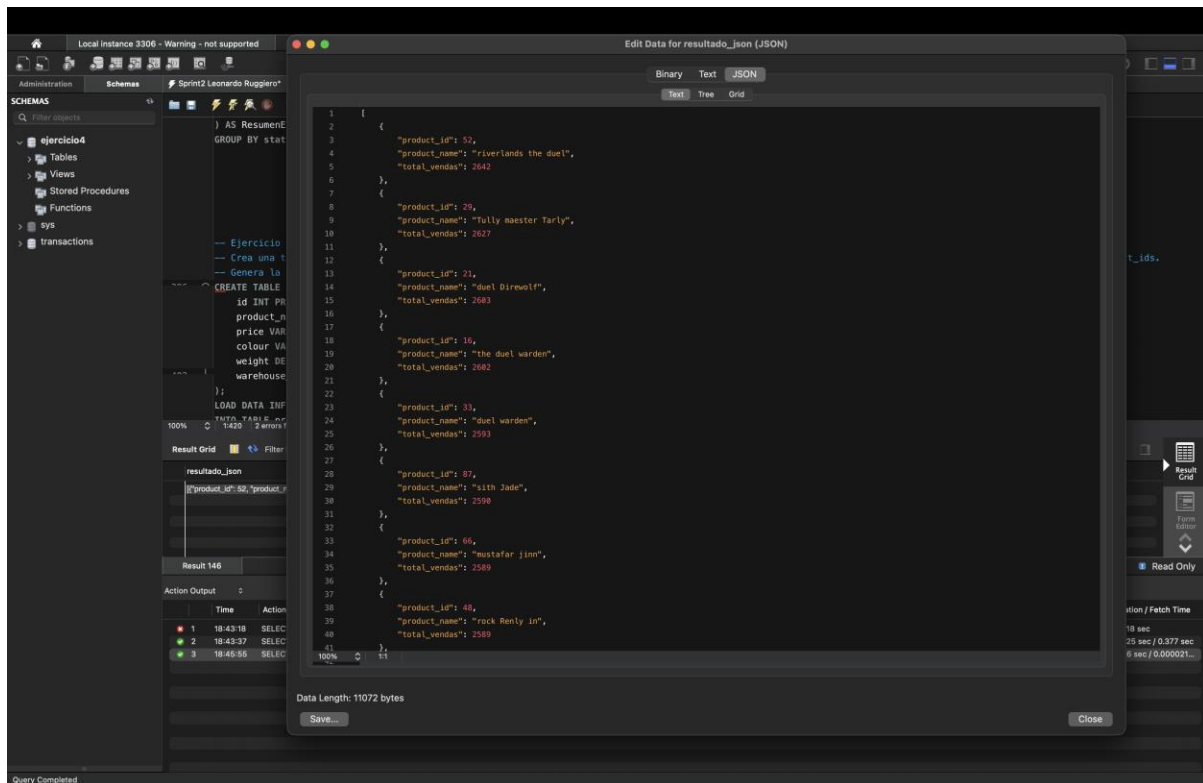
JSON_ARRAYAGG(): Con esta agrupé todas las filas en un solo array final.

The screenshot shows the MySQL Workbench interface. The SQL editor contains a query that uses `CONCAT()`, `JSON_TABLE()`, `JSON_OBJECT()`, and `JSON_ARRAYAGG()` to transform transaction data into a JSON array. The query is as follows:

```
412
413
414 SELECT p.id AS product_id, p.product_name, t.id AS transaction_id
415 FROM transaction t
416 JOIN JSON_TABLE(CONCAT('[', t.product_ids, ']'), '$[*]' COLUMNS (product_id INT PATH '$')) AS jt ON TRUE
417 JOIN products p ON p.id = jt.product_id
418 WHERE t.declined = 0;
419
420
421 SELECT JSON_ARRAYAGG( JSON_OBJECT( 'product_id', product_id, 'product_name', product_name, 'total_vendas', total_vendas)) AS resultado_json
422
423 FROM (SELECT p.id AS product_id, p.product_name, COUNT(*) AS total_vendas
424 FROM transaction t
425 JOIN JSON_TABLE( CONCAT('[', t.product_ids, ']'), '$[*]' COLUMNS (product_id INT PATH '$')) AS jt ON TRUE
426 JOIN products p ON p.id = jt.product_id
427 WHERE t.declined = 0
428 GROUP BY p.id, p.product_name
429 ORDER BY total_vendas DESC
430 ) AS summary;
431
```

The Results tab shows the execution of the query. The first result is a JSON array containing the product information and the total sales for each product. The second result is the execution log, which shows the query execution time and the number of rows returned.

Time	Action	Response	Duration / Fetch Time
18:43:18	SELECT p.id AS product...		0.0018 sec
18:43:37	SELECT p.id AS product...		0.0025 sec / 0.377 sec
18:45:55	SELECT JSON_ARRAYAGG...	1 row(s) returned	0.446 sec / 0.000021...



Una vez que terminé de ejecutar todo el código, utilicé el **Viewer** de MySQL Workbench para consultar el resultado final, ya que ahí es donde quedan guardados y organizados los datos de la consulta de una forma legible. Para hacerlo, seguí estos pasos:

En la cuadrícula de resultados, busqué la celda que contenía el **JSON** generado.

Hice **clic derecho** sobre esa celda y seleccioné la opción **"Open Value in Viewer"**.

Dentro del visor, me aseguré de marcar la pestaña **"JSON"**.

Hice esto porque, al ser un reporte complejo, el visor me permite expandir y contraer los datos para validar que cada producto tuviera su conteo de ventas correcto antes de dar el ejercicio por finalizado.